University of Economics, Prague

Faculty of Informatics and Statistics

RECRUBOT: DESIGNING AN HR CHATBOT

BACHELOR THESIS

Study programme: Applied Informatics

Field of study: Applied Informatics

Author: Denisa Nová

Supervisor: Ing. Filip Vencovský, Ph.D.

Prague, May 2020

Declaration

I hereby declare that I am the sole author of the thesis entitled “RecruBot: Designing an HR Chatbot”. I duly marked out all quotations. The used literature and sources are stated in the attached list of references.

In Prague on ………………… <u>Signature</u>

Denisa Nová

Acknowledgement

I hereby wish to express my appreciation and gratitude to the supervisor of my thesis, *Ing. Filip Vencovský Ph.D.* for his advice, support and patience.

Abstract

The aim of this bachelor thesis is to design a chatbot named RecruBot that will inform potential candidates about the company's job opportunities. This work includes well described finalized design of the chatbot, including its structure, functionalities and components. Necessary components for chatbot construction are researched and consequently discussed. The resulting design is then evaluated based on predetermined requirements and based on conducted interview with university students of various fields of study.

Keywords

chatbot, design, RecruBot, conversation agent, artificial intelligence

Abstrakt

Cílem této bakalářské práce je navrhnout chatbota s názvem RecruBot, který bude informovat potenciální uchazeče o pracovních příležitostech společnosti. Tato práce zahrnuje popsaný finalizovaný návrh chatbota včetně jeho struktury, funkcí a komponent. Nezbytné komponenty pro stavbu chatbota jsou prozkoumány a následně prodiskutovány. Výsledný návrh je pak vyhodnocen na základě předem stanovených požadavků a na základě provedeného rozhovoru s vysokoškolskými studenty různých oborů.

Klíčová slova

chatbot, design, RecruBot, konverzační agent, umělá inteligence

Table of Contents

List of Figures

List of tables

Introduction

The term artificial intelligence or AI was coined during the Dartmouth College summer AI conference in the year 1956 (Novet 2017); however, the idea of machines thinking for themselves while imitating humans or human interaction to various degrees has existed long before that.

The first imaginary creatures to be synthetic had already appeared in the ancient Greek myths of Hephaestus and the legendary Pygmalion in the form of intelligent robots such as Talos, the giant bronze automaton. Furthermore, in Greece and Egypt, sacred mechanical statues were built that were believed to be sentient, in other words to be capable of wisdom and emotion (McCorduck 2004 pp. 4-9). More recent examples are the Leonardo Torres y Quevedo's chess automaton built in 1915 (McCorduck 2004 pp. 59-60) or Karel Čapek's play "R.U.R." (Rossum's Universal Robots) that first premiered in 1921, where the word "robot" was used for the first time ever. (McCorduck 2004 p. 25).

Even though the possible advantages of the utilization of AI, such as living life more comfortably or the automation of mundane tasks, are obvious, there are naturally groups that find the idea of AI alarming. More imminent risks being that more of human labor could be automated, thus leaving workers like delivery drivers unemployed. The need for journalists to write the news would be diminished and bank services could be used just from ATMs. Nonetheless, the future of AI could also be portrayed by self-thinking machines that could once be as aware as human beings, in which one could see the potential danger to humanity; furthermore, moral dilemmas that could come with the advancement are not yet decided either.

Nevertheless, most people today imagine AI being represented by home devices like Amazon Alexa, Google Assistant or Apple Siri that very often mishear what is being told to them, therefore the notion of machines being as able as humans is being considered farfetched and infeasible at the moment. (Novet 2017)

This final thesis will be focusing on chatbots. There are many definitions of the term. The website investopedia.com defines chatbot as "a computer program that simulates human conversation through voice commands or text chats or both." (Frankenfield 2019) Another definition from techopedia.com describes a chatbot as "an artificial intelligence (AI) program that simulates interactive human conversation by using key pre-calculated user phrases and auditory or text-based signals." (*What is a Chatbot? - Definition from Techopedia* [no date]) Following definition is from botpress.com: "In simple terms, a bot is a software that can have conversations with humans." (*What is a Chatbot? Definition & Meaning | Botpress* [no date]) To summarize, all three definitions agreed that chatbot is a software that communicates with a person employing artificial intelligence.

Some of the most notable chatbots in our history are ELIZA and PARRY. From the more recent ones, notable is A.L.I.C.E (Artificial Linguistic Internet Computer Entity) that works with the exact same pattern matching technique without utilization of reason that ELIZA used in the year 1966. Despite that, A.L.I.C.E. won the Loebner Prize three times in the years 2001, 2002 and 2004. (Thomson 2007)

Even though back in the day ELIZA and PARRY used entirely typed communication to simulate conversation (Güzeldere, Franchi 1995), other chatbots nowadays include many features like web searching, games or the ability to speak and are utilized in many industries like healthcare or customer service, where they reduce the costs and increase the qualities of the offered services to society. (*How Chatbots are transforming Wall Street and Main Street banks?* 2019)

Chatbots are popular and still developing. Even today we meet them in our daily lives, be it a chatbot answering our questions about a business we are interested in on Facebook Messenger or an internal corporate chatbot helping us orient in our new employment. On the rise are also recruiting chatbots; however, still not as common as there are still many challenges that hinder the substitution of a human recruiter or a HR worker. (Joshi 2019)

In order to battle the challenges, the aim of this bachelor thesis is to design a chatbot, named RecruBot, that will inform potential candidates about company's job opportunities.

1 Literature Review

The whole area of chatbots is well covered by many publications as it is a popular topic that is developing with enormous speed as the scientific society sees future here. The same applies for chatbots – there is a sufficiency of books covering the development of chatbots from the view of developers as well as from the view of designers. This thesis draws main information from these books with the accompaniment of topic related articles found on the internet and databases of scientific publications.

The sources were searched for predominantly in English language; however, some used sources were found in Czech.

The book “*Designing Bots: Creating Conversational Experiences*” by Amir Shevat guides its reader through the process of designing many types of chatbot along with listing important decisions that need to be made. It showcases the knowledge on real-life examples while teaching the reader many basic concepts concerning chatbot design. It does not cover the actual development of the chatbot, which suited this thesis since the development is not covered; therefore, its absence made the source easier to orient in. This publication was used as a sort of recipe on how to proceed with chatbot design.

A master’s thesis “*A Warmer Welcome – Application of a Chatbot as a Facilitator for New Hires Onboarding*” by Natali Asher aims to create a HR chatbot that can offer help to new employees. Its features include help of orienting in the new offices or providing important information for creating the onboarding more comfortable. A great emphasis is placed on communication skills, which assisted with the design of RecruBot.

In the master’s thesis called “*CzechBot – chatbot pro obce*” written by a Czech author, Ing. Marek Nový, the whole process of not only designing but also developing a specific type of a chatbot is described. The author considered many practical issues from which this work took advice on, such as what areas it is beneficial to focus on the most and in what structural order.

The process of chatbot development was further explored in “Building an Enterprise Chatbot: Work with Protected Enterprise Data Using Open Source Frameworks” by a group of three authors, Abhishek Singh, Karthik Ramasubramanian and Shrey Shivam. The work focuses on design, development, maintenance and further upgrade of chatbot in corporate environment with attention to the many policies that must be regarded.

To get oriented in the field of AI and chatbots surrounding HR a document published by IBM “*The Business Case for AI in HR*” served as a guide. The authors, Nigel Guenole, Ph.D. and Sheri Feinzig, Ph.D., outline real examples of how AI delivers increased values in the Human Resources department as well as offering observations about the implemented solutions.

“*Recruiting, Interviewing, Selecting & Orienting New Employees*” by Arthur Dian is a book published for the purpose of enlightening its readers on the expertise and techniques of the most successful recruiters. Some chapters describing the practices and giving advice on how to earn the attention of potential candidates were used as an inspiration in designing the RecruBot’s dialog and character.

2 Problem Analysis

This chapter discusses the topic of chatbots and focuses on the main fields that need to be covered in order to design a chatbot. The purpose of this chapter is to summarize and highlight the areas that need to be considered to fulfil the aim of this work: Designing a chatbot, RecruBot, which will inform potential candidates about company's job opportunities.

Initial and crucial, when one wants to design a chatbot, is to determine what type of chatbot they need. There are chatbots that communicate with more people at once, team chatbots. Personal chatbots, on the other hand, lead a conversation with one person at a time. Since this paper is aiming to design a recruiting chatbot, there will be no need for other participants than the protentional candidate and RecruBot itself, hence it will be a personal chatbot.

Next, it is wise to understand if the needs will be better realized by designing a superbot or domain-specific chatbot. A superbot is well represented for example by Google Assistant, which not only responds to the inquiries by his learnt responses but also by incorporation of other services such as Google Maps, weather, news and so forth. A domain-specific chatbot serves just one purpose; thus, it is usually well specialized in its one utility. (Shevat 2017) Furthermore, the chatbot's personality can be more easily designed to suit the image of the company it represents. A domain-specific chatbot clearly befits how RecruBot should be perceived.

RecruBot must be capable of understanding human language and use natural language to respond back. In other words, it needs to be able to respond based on full sentences people use to communicate without the need of the user to simplify his input in unnatural way. In addition, its responses need to be in full sentences that would not feel odd or robotic. It needs to be able to discern the topic of the message. This is to be accomplished by teaching the chatbot to recognize intents and entities of the message content like the line of business, salary or starting date. This will help the chatbot to understand questions like "Are there any open jobs in Prague?" since on top of recognizing a request of listing open positions, it must be able to identify that the open positions to be listed should be in Prague. In case the chatbot would not be able to process the message, it must be able to inform the person so and direct them in conversation or to refer them to a human operator. The ability of processing natural human language will make the chatbot feel more humanlike and make the user more comfortable while chatting with it.

RecruBot must be able to answer queries about the job offers. As the chatbot's main purpose is to demonstrate and advertise the company's job opportunities, it is vital for it to have the capabilities to answer questions about the job openings like "What kind of benefits do you offer?". Moreover, most human conversation is not on a ruled schedule of question followed by answer followed by another question, therefore the chatbot should be able to try to develop a conversation by asking its own questions like "What is important for you in your job?" or "Are you happier working on site or online?" and similar conversation starters that could lead to a more effective advertisement of the company's openings. This will also make the conversation more memorable as it will prolong the interaction and make it more impressive.

Since RecruBot's purpose is to inform visitors about available positions, it should always be well updated on their description and their status. This could be, of course, ensured by changing the chatbot's dialogue in its source code all the time, but more practical way would be including an interface for simple database connection from where the actual positions and their information would be loaded and presented. Another great benefit to using a database is the extra space for maintaining logs, in case the chosen platform would not provide satisfactory log services. Thus, RecruBot should be designed to have access to a database, where all needed data would be saved and always available. In addition, to this requirement is connected the simplicity of integration to more systems as the software should be able to communicate through various network protocols of third-party systems, as well as employ various types of API. Altogether, connection to a database ensures easy information updates, accessible logs and easier portability.

Another requirement is for the chatbot to have a web user interface ready to use, since its main purpose is to represent and inform about a company. Thus, the option to embed the chat client into a corporate official webpage would be crucial.

Even though one might try not to, we judge people based on their first impressions and changing one's initial opinion is not easy. The same applies to corporations. RecruBot will essentially represent a company, therefore its way of interacting needs to be respectful, polite and without grammar mistakes in order to create a first impression of a trustworthy company. The chatbot must introduce the company, itself and all the job opportunities in a coherent way, so that the potential candidate would have a clear idea of what the chatbot is offering. Furthermore, the chatbot's personality must be interesting, in order to make the visitors remember their visit to the company's site. To conclude, last demands are for the chatbot to support positive image of the company and to leave a positive impression.

The RecruBot will be designed and consequently judged based on the following requirements. Furthermore, this thesis's aim will also be reached if all the requirements will be properly met.

Table 1: List of initial requirements

id	Requirement name
1	Responding in natural language
2	Understanding natural language
3	Recognition of the intent
4	Ability to extract entities
5	Ability to inform about the open positions, the company and itself
6	Database connectivity
7	Accessible logs
8	Simple portability
9	Web user interface
10	Pleasant corporate image
11	Interesting personality

3 Analysis of the current situation

This following chapter dives into the knowledge that is crucial for choosing the suitable elements for designing a chatbot. The main topics have been derived from the previous chapter. Firstly, the present possible solutions to recognizing and processing user input will be explored. Next will be examined the architectonic types and their options of utilization for chatbots. Lastly, there will be an overview of how to create the chatbot replies in a more immersive manner that would make the chatbot feel more human and pleasant to communicate with. All these areas are important to get oriented in before the actual designing of a chatbot.

3.1 User input processing

Chatbots usually obtain user input in textual form; even vocal chatbots to which you can also respond by speaking use a speech-to-text feature before processing the input. There are three basic ways they do that. The easiest one is to let the user pick only from predefined answers and questions. A slightly more complicated option is to let the chatbot pick out keywords and return its response based on these. The most complicated but also most powerful is to process the user input by a system for Natural Language Processing (NLP). We can thus differentiate types of chatbots depending on the technology they use to process user input.

3.1.1 Chatbot types based on input processing

Chatbots are divided into three categories. Each of the category has its own advantages and it would be wise to explore them in order to decide, which type of chatbot one needs to design. The types are:

1. Menu/button based Chatbots
2. Keyword Recognition-Based Chatbots
3. Contextual Chatbots

Button based chatbots are the simplest from the three that are used in our society. One can imagine them as properly done decision trees through which the user can travel thanks to the predefined responses that can be chosen. These responses are usually displayed as a set of buttons below the on-going dialog; by clicking them the dialog will progress. Although this kind of chatbot may be the solution for answering 80% of the frequently asked support queries, and therefore saving great amount of time (Phillips 2018), it is quite time-consuming to get to the deeper issues as even more trouble comes when trying to solve more complicated queries. This kind of type is easiest to implement; however, it also does not leave anyone in awe of the technology, which could lead to evaluating the company the chatbot is representing as basic or not future-oriented.

The second type are the keyword recognition-based chatbots. These are more advanced than the previous type as they can listen to what the user is saying. These chatbots look for customizable keywords to evaluate the intent of the message. For instance, when a user asks, “How to bake chips in the oven” the chatbot will pick out the keywords “bake”, “chips” and “oven”. From these three words the intention of the question will be clear. It is common practice to combine button-based and keyword recognition-based chatbot due to the possibility for a user to use certain predefined questions after their own attempts have failed. (Phillips 2018) These kinds of chatbots seem more progressive, therefore it should not have negative impact on a company’s image; however, it might happen that the user would not get their intent across in case he would not use the expected keywords. This especially becomes a problem if the company is international, meaning even non-native speakers use the chatbot and non-native speakers are more likely to ask a question in a non-standard way, avoiding the expected keywords.

The third and last type, the contextual chatbots, are the most advanced of the three. They employ Artificial Intelligence to remember past conversations and, with the help of Machine Learning (ML), improve themselves to respond better next time. For example, a chatbot supporting pizza ordering could remember the user’s last order, making it easier for the user to order again. In addition, these chatbots usually utilize natural language processing, thus they are taught to recognize the intent of the conversation without relying on the keywords, instead focusing on the use of entities. (Phillips 2018) In this case, it is the most impressive one as far as image is concerned; furthermore, the issues the other kinds of chatbots can be eliminated. On the other hand, it is the hardest one to develop, but this difficulty could be avoided by using a platform that incorporates NLP. All in all, the ability to recognize intent without the need of predetermined keywords would make it most probably the best fit for the purpose of RecruBot, since these days there is a considerable amount of competition in most fields and every advantage is welcomed, even a small one such as an advanced functioning HR chatbot. Because of these reasons the contextual chatbot best suits the aim of this work.

3.1.2 Natural Language Processing

“Natural Language Processing or NLP is a field of Artificial Intelligence that gives the machines the ability to read, understand and derive meaning from human languages.” (Yse 2019) The last type, contextual chatbots, use this technique to make their interactions more progressive. For effective utilization of this technique, it is important to understand how it works and its capabilities.

To simply explain, NLP enables the machines to understand humans without the human using Java or some other programming language; it creates a bridge to make the communication easier as the machine can obtain and process data from written or verbal messages. Most notable differences between NLP and common word processing is that NLP does not only search for keyword but analyses the hierarchical structure and finds out intent from considering attributes such as tone, structure of the sentence, context and idioms.

(Jassova 2020) This is possible thanks to pre-programmed procedures or knowledge acquired through experience. For instance, when the user asks, “What experiences would I need?”, the chatbot is able to connect the question to the concrete job offer that was discussed earlier.

The main techniques (Goyal, Rattan 2017) to analyze the language are:

- Morphology
- Lexical
- Syntactic
- Semantic
- Discourse
- Pragmatic

The syntactic and semantic analysis is essential for a chatbot, since in there belong the practices of extracting named entities and text understanding. Named entities are specific words that describe real life objects with a concrete name of anything from locations through persons to products. (Nadeau, Sekine 2006) This is the foundation of making the chatbot sound natural, which then helps to convince the users to spend time with the chatbot. More time of conversation with the chatbot leads to more exposure to the content the chatbot is advertising; hence, better fulfilling its purpose.

NLP can be divided into two categories (Goyal, Rattan 2017) based on what approach is employed while processing the language. In the past, it was common to use predefined rules of recognizing the content of a sentence. However, this practice was arduous. The second approach is to depend on machine learning; through algorithms, enormous data of text was analyzed and condition models for determining the intention were created. The results of these two practices can be unrecognizable depending on how well they are implemented. Today, developers are not forced to compose their own technique of language processing from the ground up as there are many platforms that have a good basis of the processing already implemented.

The enormous power of NLP is better understood when one is introduced to some examples of its use. NLP powered software help people to protect their email folder from spam; NLP Group at MIT developed a system to help with the identification of fake news and many of us are already using the virtual assistants like Amazon Alexa or Apple Siri in our everyday lives. The most impressive, however, would be the use of NLP in the healthcare industry. The technology helps with faster diagnosis, decreasing costs or improving customer delivery. Thanks to NLP, an upcoming disease like cardiovascular illness, depression or schizophrenia can even be predicted, thus enabling the healthcare system to take proper preparative steps to reduce the possible upcoming risks. (Yse 2019)

There are many great challenges that NLP must still completely defeat since the nature of human language is so intricate. Some of the rules to human language are very abstract so detecting them is hard. One example being the usage of sarcasm or hidden meanings that are very often used in communication of humans, who can generally easily grasp the ambiguity of a language. (Garbade 2018) In spite of these challenges, NLP is presently a very quickly growing field of development and study; one can observe this reality simply by exploring its many uses and hopes for the future.

3.1.3 NLP solutions

It has been established that chatbots using NLP have the highest chance of imitating a close to human interaction, leaving the strongest impression and effectively providing information. For these reasons it will be employed in the design of RecruBot; however, as there is little sense in creating something that already exists, different solutions of implementing NLP need to be inspected before one can be chosen.

There exist an incredible number of environments where one can develop its own chatbots. Some platforms do not even need any programming: they offer graphic flow editors and beautifully done analytics dashboards. Other support more advanced tools for developers including APIs, SDKs, IDEs. The popular ones even provide step-by-step guides on how to proceed with the building of a brand new chatbot. Many of these even grow communities of enthusiasts that help others and share their ideas and their discoveries. (Monsters 2019) One can even find platform that will create our own unique chatbot with the ability to speak using the chosen specific voice and with the exclusive personality that will best fit our brand. One such environment could be the support of RecruBot.

> Since there are so many options, it is quite difficult to choose where to integrate a chatbot. An article by Data Science Learner recognizes these five cloud platforms (*Top 5 NLP Chatbot APIs to Make Your First Conversational Chatbot* 2018) to be the most fulfilling tools.

- IBM Watson Assistant
- DialogFlow
- Amazon Lex
- Wit.AI
- Microsoft Luis

All the listed solutions above belong to the category of SaaS, software as a service, meaning that the functionality is provided as a service via internet, while the provider runs the applications on their own servers. This is usually accompanied by the need of giving the permissions to process the data that flow through. An alternative could be Rasa, an open sourced framework containing all tools to create a powerful NLP incorporated chatbot. Rasa is an on-premise solution. On-premise solutions have been around for longer and are still well trusted, and in some situations, more sought after; however, in the case of running a chatbot that is supposed to be easily integrable to different platforms a SaaS solution would be more acceptable. Furthermore, SaaS solutions tend to be generally less expensive, which adds attractivity to chatbots designs that are meant to be used by corporations as RecruBot is.

With the exact same selection of "best" SaaS NLP chatbot solutions agrees an article "Natural Language Platforms: In-Depth Guide [2020 update]", of the blog AI Multiple that is known for administering business consultations in the field of B2B AI market. A brief overview of each, highlighting their strengths and weaknesses, will help in future decision of which would be most beneficial to integrate the chatbot in.

IBM Watson Assistant

Watson Assistant created by IBM has the greatest advantage in the fact that it supports many channels, from mobile devices through messaging platforms to robots. Using Watson Assistant, it is possible to create natural language understanding application that is able to learn from previous experiences. Moreover, it supports Machine Learning base entities recognition. It can also easily connect to various messaging channels, web environments and social networks. It the most corporation aimed from all five, which mean it could be well suited for the purposes of RecruBot. Furthermore, it supports more than 10+, even though, most of them are still in BETA.

Dialogflow

Dialogflow is a Google service that is running on Google Cloud Platform. It can create speech to text and text to speech that is powered by machine learning. It can be integrated to the majority of dominant platforms like Facebook Messenger, Slack, Alexa and Google Assistant. Its support ranges from laptops to cars. Dialogflow currently supports more than 15+ languages. The portability of RecruBot would get much easier if DialogFlow would be used.

Amazon Lex

Amazon Lex provides the deeply complicated functionality of NLU (natural language understanding) and ASR (automatic speech recognition). Amazon Alexa runs on this very engine and now any developer can obtain access to this technology. Both text and voice are supported, although only language supported is English. However, the support of integrating into common chatting platform is not as great as with DialogFlow or IBM Watson Assistant.

Wit.AI

Managed by Facebook, Wit.AI is a conversational chatbot APIs that are easy to integrate into other devices or applications. It supports 50 languages and its use is completely free, the only condition being the need to notify the provider in case the queries are going to exceed 1 request per second. Even though one retains the data ownership, the intents, entities and validate expression are accessible by the community while one's app is open. Besides the typical interfaces, it supports also wearables and home devices. Unfortunately, once again the support of integration to popular messaging platform is not that impressive as with other platforms in this list, which makes it not the most suitable for the purpose of RecruBot.

Microsoft Luis

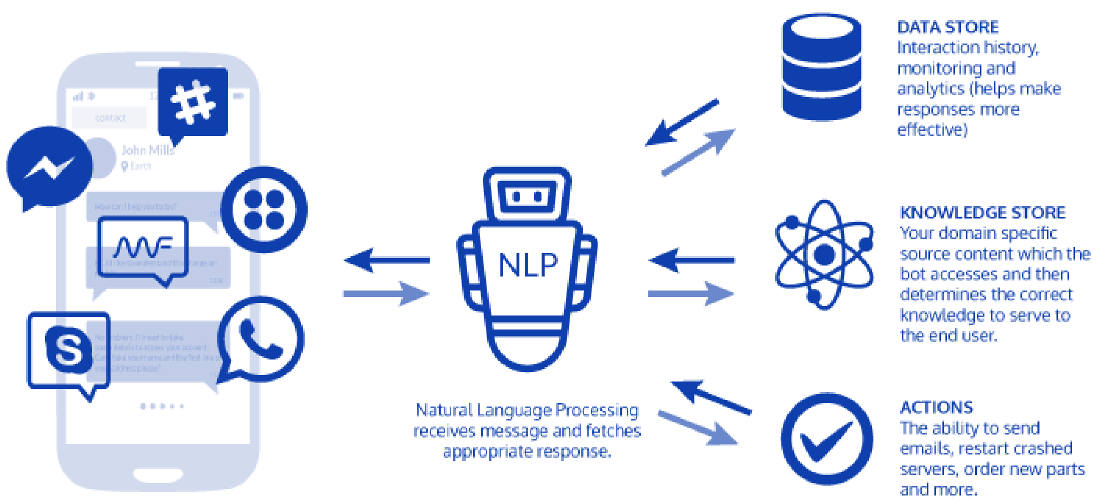
MS Luis is a service supporting both the voice and text messages. Its purpose is to quickly create enterprise ready but custom models with the possibility to improve. It offers useful features for Azure integration which is not surprising as its provider is Microsoft. (appliedAI 2018) However, it does not support integration with common chatting application like Facebook messenger or Skype. It offers advanced NLU techniques, but it is missing other important the option to create such conversational logic. To create, one would have to integrate with, for example, Azure Bot. Furthermore, this solution requires much more coding experience when compared to the other solutions. Therefore, it might not be well suited for the purpose of RecruBot.

3.2 Architecture

Most used computer software is not functioning by itself but communicates with other software components in order to accommodate all needs of today's systems. Same applies to chatbots. In this chapter it is explored what kind of architecture does a chatbot need, which is a knowledge that is necessary for designing a new well-functioning chatbot.

To grasp the general concept, let us first understand a basic architecture. Figure 1 is a very general scheme of how a chatbot works.

Figure 1: Basic general chatbot architecture (*A Breakdown of Chatbot Architecture And How It Works* 2019)



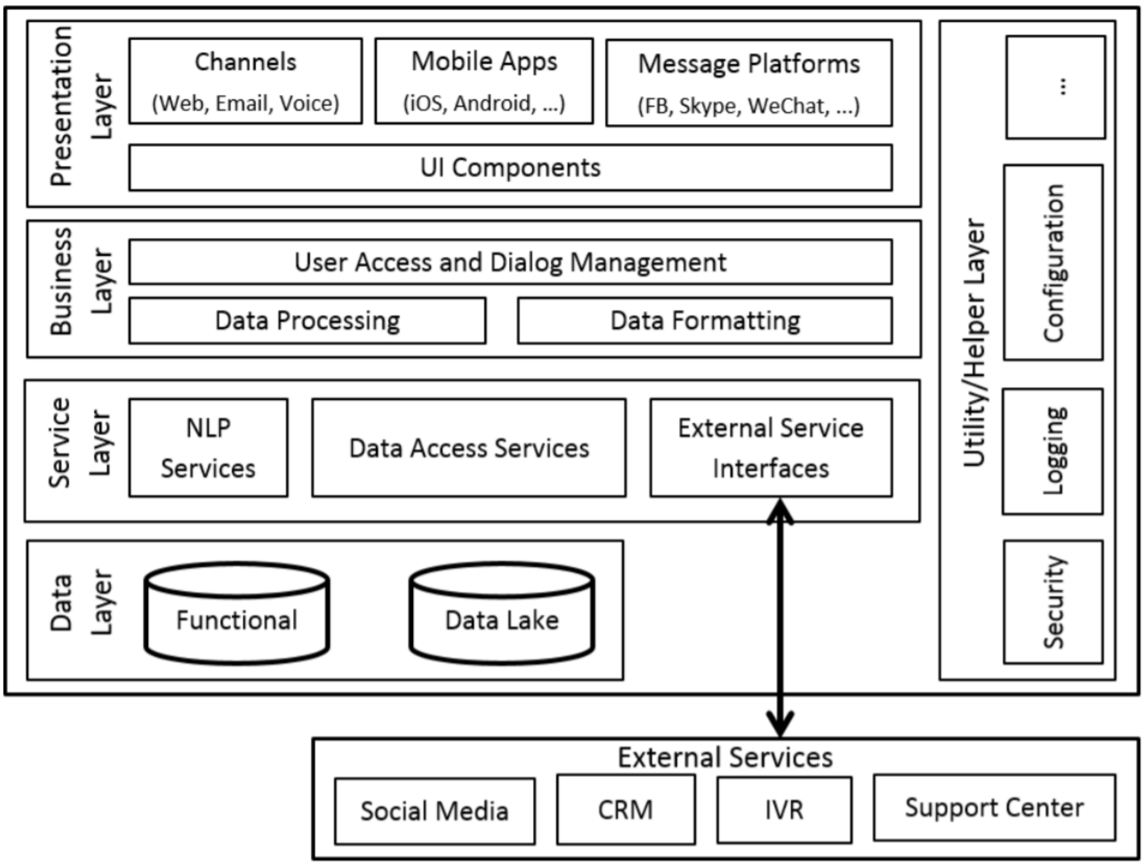
The phone represents the many mediums the user can use the reach the chatbot. From the medium then travels the input to the NLP that analyses it and extracts needed intent, possible entities and context. Based on this information and predefined rules, the best suited response is retrieved from the knowledge store and sent back to the user. In the knowledge store are saved the data to answer queries. It could be a library of FAQs and their answers or a list of open positions and related information. All history of conversations together with available analytics are stored into data storage.

A more in-depth look on the chatbot architecture is taken in the article "Standardized Architecture for Conversational Agent" from the International Journal of Computer Trends and Technology, where the author, Roshan Khan, divides it into four main layers (Khan 2017):

- Presentation Layer
- Business Layer
- Service Layer
- Data Layer

The architecture's organization is shown in the diagram in figure 2. The basic summarization would be basically identical to the previous figure's one, so to understand more deeply let us inspect each layer more closely.

Figure 2: Typical Chatbot Solution Architecture (Khan 2017)



Presentation layer

Presentation layer is operated by the users with the help of various media for interacting with the system. It is simply the many faces of user interface. It is divided based on the type of interaction from the user, where is the interaction's origin and through which platform it comes. The user can send input through various channels; examples being chat on website, social networks, email or voice input. Very similar support is to be done for platforms as users would not appreciate limitations in this area either; support of mobile platforms like Android or iOS or web platforms like Facebook Messenger, Skype or WhatsApp have all different properties. Their constant development should be followed as well as keeping in mind this fact when designing this part of the chatbot architecture to ensure it is flexible. The input travels from the presentation layer to the business layer.

Business layer

Business layer is responsible for data processing which is usually separated into a few consequent steps that transform data from the service layer to the business entities that represent the actual topic objects like specific job position, specific requirement et cetera. Data formatting takes place on the input as well as on the output since different platforms in the presentation layer require and support different data formats. For instance, Facebook Messenger supports the sending of an image while sending an image using Google Home or Amazon Echo is impossible (Khan 2017). In addition, part of the business layer is the most important component of chatbot, a dialog manager. This component tends to the context of the underway dialog and processes the user interaction The most important task of dialog manager is to decide the next action of dialogue system according to the current context. (Jurafsky, Martin 2019). One could say that the creation of the dialog manager holds the most value in the potential future success of a chatbot; thus, a great focus should be retained while designing this piece of software.

Service layer

The importance of the service layer is immensely increased by the included NLP component that, as is described in the previous chapters, contains the function of processing along with understanding of the text that is written in natural human language. The whole functionality is often based on these abilities; therefore, it is important their correct performance. Moreover, a part of this layer is an access to data that are responsible for the connections to the other services, from which it can retrieve data or send data to. Similar function is held by a component for access to external services interfaces, through which one can use, for example, diverse APIs.

Data layer

The data layer is present primarily due to the many components of the other layers that need to store and load data. It is vital for a chatbot to have a quick access to this layer because the majority of tasks communicate with this layer; in other words, the speed of the service is strongly dependent on this layer. (Khan 2017) Reporting data, conversation analysis data as well as other functional data can be found here.

In the diagram it can also be seen a fifth layer, a helper layer, which even though is not considered to be a one of the crucial ones; however, it also has its quite important uses. (Khan 2017) This layer ensures the security, different needed configurations, logging and other features that can be implemented.

Now that it is revealed how a well-done chatbot architecture should be organized; it becomes possible to design one while concentrating on the parts that are essential for the good functionality and responsivity of a chatbot.

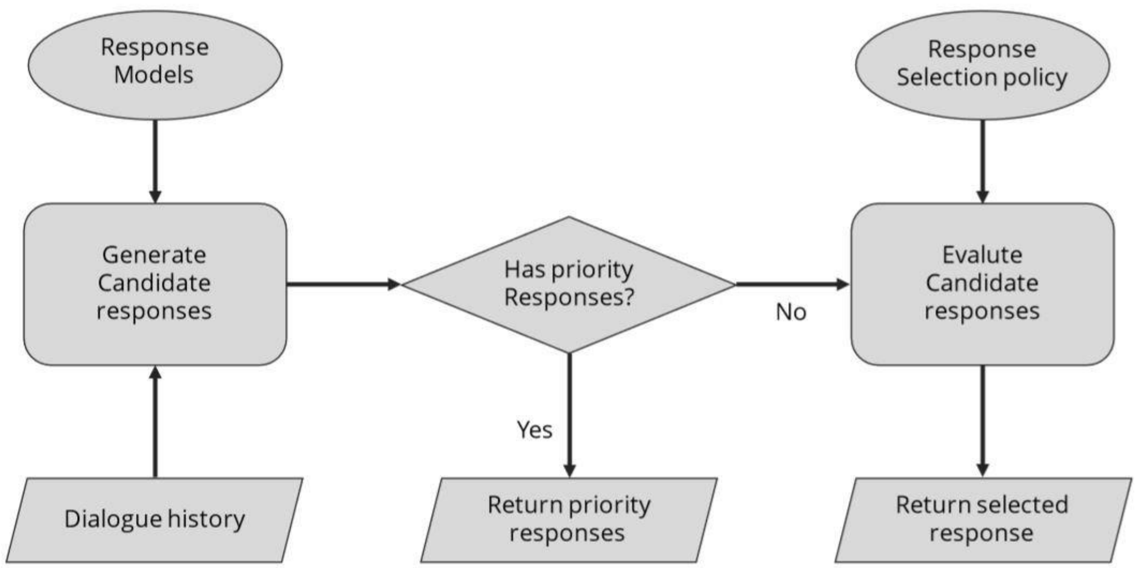
3.3 Response generation

Since a chatbot is essentially a conversation robot, response generation is one of its vital parts. Thus, the knowledge of how to ensure its proper process is fundamental; this includes not only how to technically ensure the generation of comprehensible text for the user based on the input, but also how to graphically display the response to the user. Furthermore, the technique of how responses are styled is also included. Also, since simple portability is one of the requirements of the final design, the way of how to integrate into webpages is part of this subchapter.

3.3.1 Dialog manager

A dialog manager starts to work after the user input is processed by the NLU component. From the procession results the dialog manager receives the recognized intent and entities that are already learned. Dialog manager then generates a group of possible responses while taking into consideration the intent, entities and dialog history of the conversation. More advanced dialog managers can also have set priorities of their possible responses that will play a role in deciding which response is the most appropriate to send. This logic is well demonstrated by the diagram in figure 3.

Figure 3: Dialog Manager Control Flow (Dasagrandhi [no date])



The process of looking for fitting response is best imagined as a decision tree with each node holding responses for different intents. After dialog manager finds the correct node, it then searches for best response base on the entity. For example, if a user writes "I am looking for a job in IT", the dialog manager recognizes the intent as #looking-for-job or similar predefined equivalent, the entity is identified as @IT. Thereupon, the manager knows the user is asking for jobs in the IT department and can provide the fitting answer. The dialog manager is as smart as the many intents and entities he is taught to recognize and react to.

The SaaS solutions providing NLP that were discussed in section 3.1.3 also provide a dialog manager component that is based on predefined rules. The advantage of these solutions is that they have directly integrated NLP component, hence there is no need to solve the communication between the two components. They usually provide the ability to create rules programmatically using the API or edit their rule declaration file. But some, like IBM Watson Assistant, also provide a synoptical graphical user interface that allows non-technical users to edit responses, intents and entities, which greatly simplifies development of new chatbots.

Dialog manager is a chatbot component that will need a lot of attention while designing a chatbot as all its abilities are defined in the response tree's nodes.

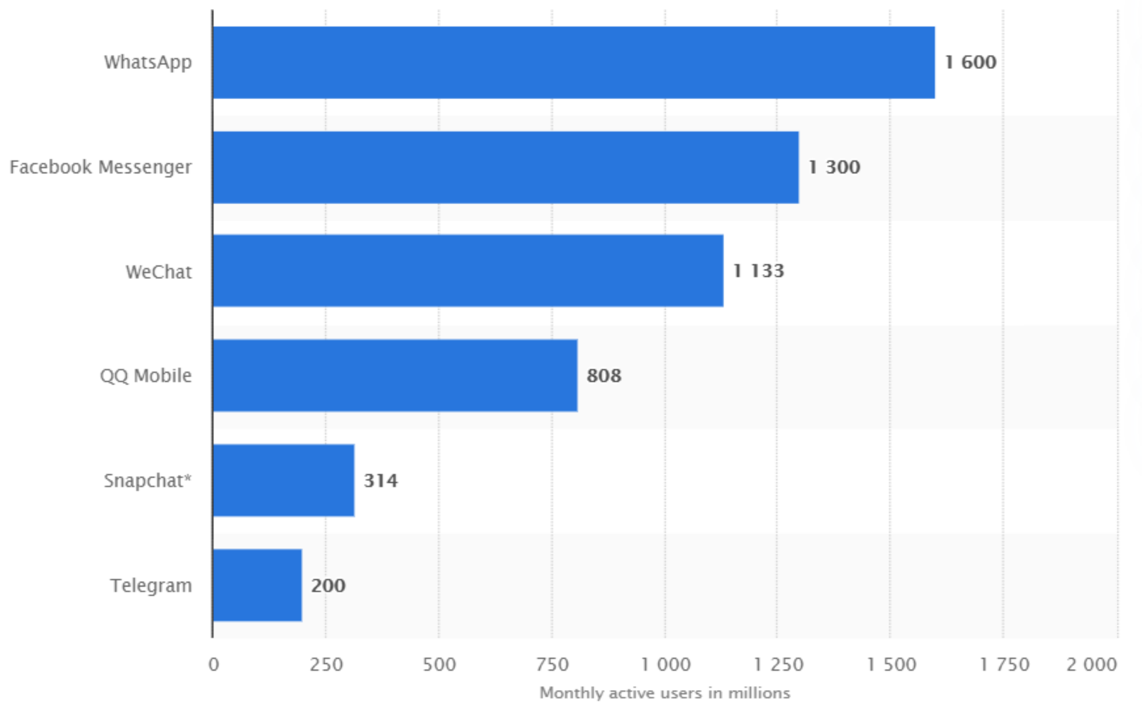
3.3.2 User interface

User interface is generally the tool which enables a user to interact with a machine. In the case of a chatbot, the user interface is a medium through which a user can send a message that arrives to the system of a chatbot. Usually the message is a text; however, a voice input is also possible. Voice chatbots use speech-to-text converter, so in the end, the software still processes a written input.

A user interface is usually a chat platform or some messaging app, where the user communicates with the chatbot via written messages or predefined responses that are shown as clickable buttons in the chat window. A user input can also be in a different form than just text. If the chatbot has implemented corresponding features, it can have the ability to react to images, shared location or a file. For example, while asking the question "how do I get to your office?", instead of the user being forced to describe their location, they can simply share their location using an API; consequently, the chatbot will react by generating directions for the user.

Figure 4 shows the six most used messaging applications in the world.

Figure 4: Dialog Manager Control Flow (Statista 2020)



Besides these chat platforms one should also consider separated chatbot on the website that does not perform the main function of the page. The chatbot serves as a complementary function, for instance, as a quick way of communicating with support for the service that the website promotes. Chat usually takes the form of a small expandable window on the right lower edge of the screen.

Since portability is one of the requirements for the successful design of RecruBot, it would be wise to ensure easy integrability to as least a few of the most used application listed above in figure 4, together with webpage integration support. Great advantage is that most of the popular messaging platform have a good support of integration either by the platform the chatbot is being developed in or by the messaging platform itself. Same cannot be said about the integration into a webpage.

There exist services for the web that provide a usable web component, which usually provides integration with other commercial services. However, the question then is, if it is not just another conversation platform, as most of the messaging platforms above provide a web interface and some of them, such as Facebook Messenger, integrate into websites.

A frequent approach is the creation of a custom customer chat, which is either placed on its own separate page or placed in the bottom corner of the page as a button that expands the chat window when clicked.

Web technologies are not restrictive for creating such widgets. A WebSocket can be used for data transfer to ensure that new messages are pushed. WebSocket also has the advantage of providing asynchronous communication. This keeps the TCP connection open and does not have to establish a new connection every time we want to send data, as is the case with AJAX technology. (Nový 2019) The web widget should be easily added to any king of webpages. After creating the web widget, the integration to the actual webpage is only a question of adjusting the appearance of the webpage using the standards of HTML, CSS and JavaScript.

The possible portability would highly increase by creating RecruBot as a web widget, considering the final widget would be enclosed, so that it could be easily added to any needed webpage as a package. Also, the possibility of RecruBot being at the webpage of the company it would represent would positively influence the company's image by making their webpage more interesting. In addition, the amount of people exposed to the chatbot would also increase, which could lead to more potential candidates applying for the open job positions.

3.3.3 Dialog design

As chatbots are currently still on the rise, there is no generally accepted and well tested out approach that would lead to a designed chatbot dialog one desires. Because of this, there are many pieces of advice and approaches from different sources that agree on certain things but differ on other. A few of the sources even predict that soon it will be normal to see a job positions called "conversation designer" (Black 2020), which would be a person specially hired to design dialog flows, write chatbot responses and all in all create the whole chatbot personality to make the conversation as unforgettable as possible.

Even though the sources do not agree on all the steps and advice, there are points that they share. The shared guidelines were considered, and the missing spaces were filled out by either a combination or by the ones that seem the most reasonable to the author of this thesis. As a result, a basic guide on chatbot conversation guide comes to fruition.

The steps are as follows:

Step 1: Decide on the bot's purpose

Before starting to create any dialog, it is important to clearly define the chatbot's purpose. Is it simply a chatbot for information on the weather? Is it a guide in one's journey to a healthy lifestyle or a dream figure? Or is it supposed to be a constant companion in one's life, reminding one of his previous engagements while booking his doctor appointments?

After answering this question, it is strongly recommended to list all its duties. To share an example, let us imagine we are designing a chatbot whose name is Olivia and its purpose is to help customers with their choice of olive oil; then its duties would be:

- Welcoming the customer on the olive oils e-shop
- Introducing the company selling the oils if asked
- Informing the customer about any oil that is offered
- Making recommendations based on which cuisine the customer is planning to cook

By establishing this list, it is suddenly much clearer where the conversation should lead.

Step 2: Create bot's personality

Before starting to write any conversation, a successful engaging chatbot needs a personality. Depending on the personality, the diction, tone and mood of the conversation will vary. One source even recommends constructing a backstory for the chatbot. (Liebl 2019) That is a technique used in creative writing and the idea might produce results. To demonstrate, if a person was asked to write a story about a man, they would not have many ideas what to write about since it is shallow and too general; however, when a backstory would be built, the assignment would suddenly change to: Write a story about a ginger warrior facing his own father in a battle. The ideas suddenly keep flowing in.

The same, in theory, should work for the chatbot. By creating a backstory, even a simple short one, the chatbot's persona becomes more apparent. After details, like if the chatbot should behave more like a servant, equal or perhaps a dominant one, will be added, a unique chatbot persona is born.

As for an instance, Olivia's backstory could be a middle-aged Italian woman that loves cooking; She is a very heart-warming person that knows nothing but kindness. From this short description, it is already much easier to imagine how such a chatbot would communicate. It would always react amiably and with compassion. In addition, it could respond passionately to some cooking consultation.

Of course, it is also necessary to realize that a chatbot communicating with a lot of emojis and using lot of slang might not be a good representation for a corporate chatbot, like RecruBot is going to be. Furthermore, in some cases a robotic straightforward chatbot might also be a goal. On the other hand, even a corporate chatbot does not have to mean a boring persona.

Step 3: Build a conversation diagram

In this step the actual creation of the conversation begins. The purpose of the chatbot had been stated, so we know where the conversation should lead. A conversation diagram are all possible outcomes of the dialog mapped out with the help of conversation elements. It could get very complicated if a targeted chatbot is too complex that is why it is advised to start with simpler diagram and build on it, alternatively to make more than one.

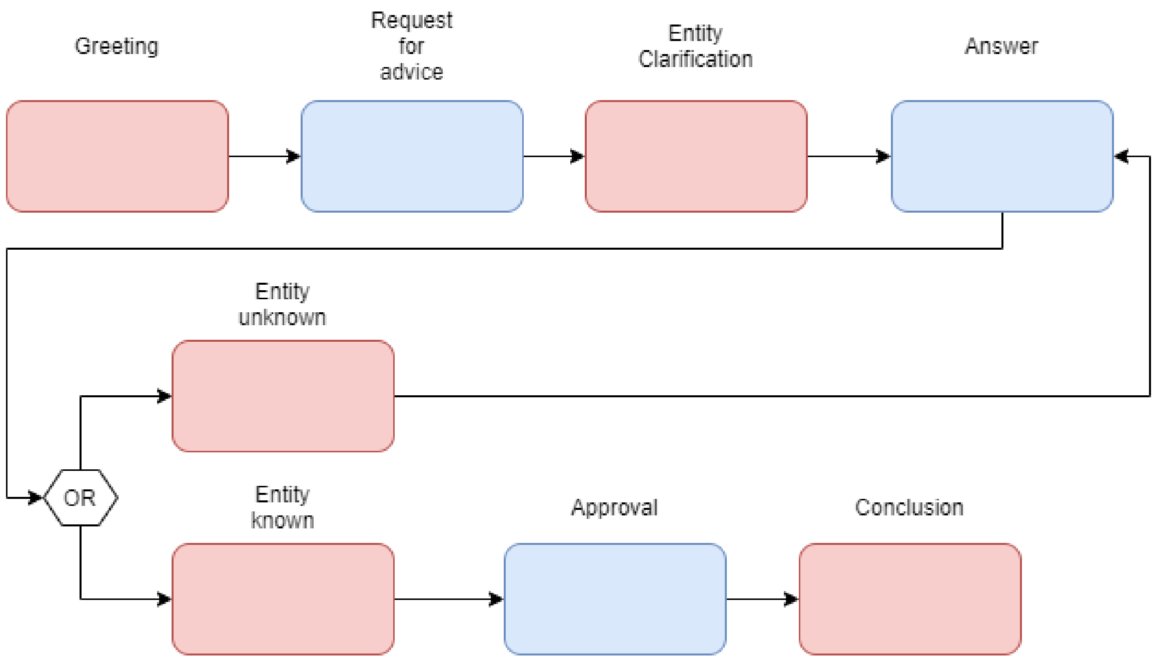
Conversation elements (Liebl 2019) to be used are:

- Greeting
- Asking
- Informing
- Checking
- Error
- Apologizing

- Suggesting
- Conclusion

A simple example of a conversation diagram for Olivia can be seen on figure 5, from which it can be observed that the responses from the bot and user take turns as is usually the case with chatbots. The red squares are Olivia's messages; the blue squares are the user's messages.

Figure 5: Conversation diagram - Olivia, olive oil advice



A conversation diagram is an excellent method of designing a conversation flow without being disturbed by thinking of what the chatbot will say, keeping the designer focused on the logical aspects of the dialog.

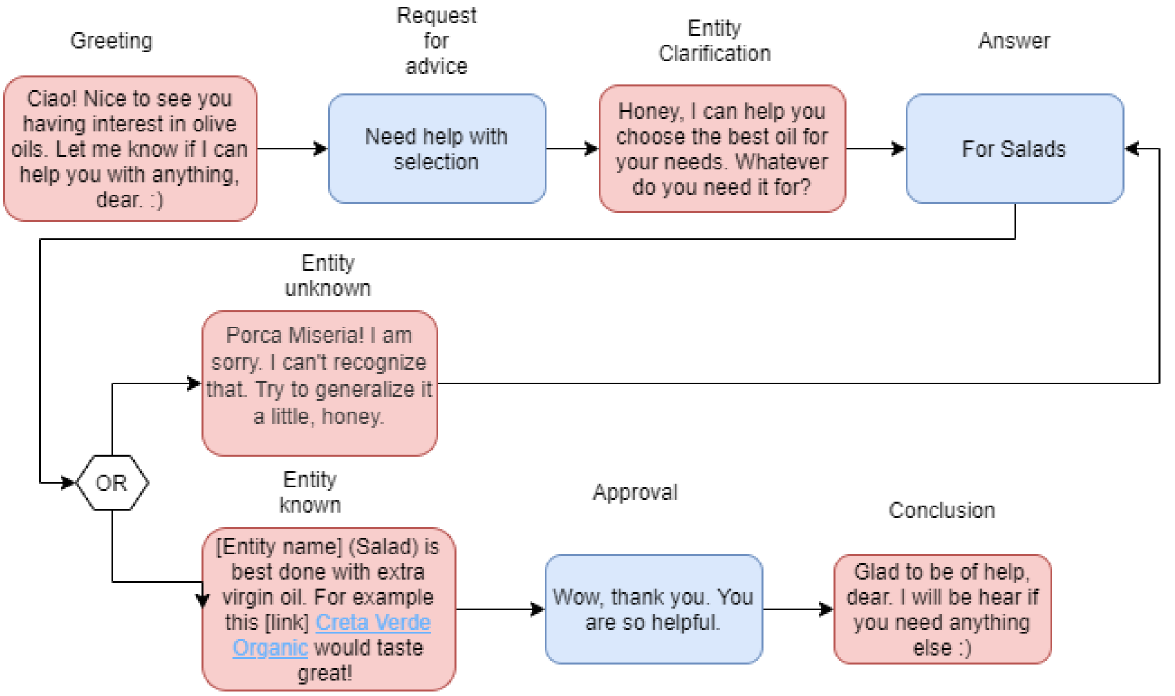
Step 4: Fill in the diagram with dialog

The logical structure of the conversation is done; now what remains is to fill it in with concrete worded responses of the chatbot. In this step, it is essential to remember the chatbot's persona.

In addition, since it is a chat the response should reflect that, in other words, the responses should be to the point; No one would be happy if they received a 300 words long answer to a question of a type "Will it be sunny tomorrow?". Moreover, imagining of being the user and not knowing the details about where the conversation should be leading will help the designer to ensure the responses are meaningful and clear.

How such a filled in diagram might look like is shown in figure 6.

Figure 6: Filled in conversation diagram – Olivia, olive oil advice



After rewriting and examining the conversation flow, all that is left is to test it. This can be done by using a software like Chatfuel that lets anyone create a simple conversation chatbot on Facebook Messenger or on any platform on which the chatbot for which this dialog is being designed.

3.4 Conclusion

In summarization, in this whole chapter were covered all the areas that are needed to design a full chatbot solution. Generally well-received chatbot platforms were introduced and evaluated in order to select a platform for RecruBot. Common functioning chatbot architecture was explored since an architecture is one of the outputs of design; thus, it would be irresponsible to attempt designing it without any prior research on any approved approach. Next the knowledge of how chatbot generates responses, the importance of dialog manager and its options of response selection customization were explored. The potential methods of integrating the chatbot and popular interfaces were discussed, so that the chatbot's possibilities of portability would uncover. In the end, the guide to designing the hearts of chatbots, the dialog, was assembled. All this knowledge will support the following designing of RecruBot.

4 Methodology

In this chapter is explained how and why a slightly modified waterfall model was chosen as the methodology for this work. Then the methodology's details are introduced. However, since the aim of this work is to design a chatbot and not its whole construction process, the progress will finish by completing the design phase. Furthermore, to make this thesis more valuable the design stage will proceed based on the chatbot design process constructed and recommended by Amir Shevat in his publication "Designing Bots, Creating Conversational Experience" (Shevat 2017).

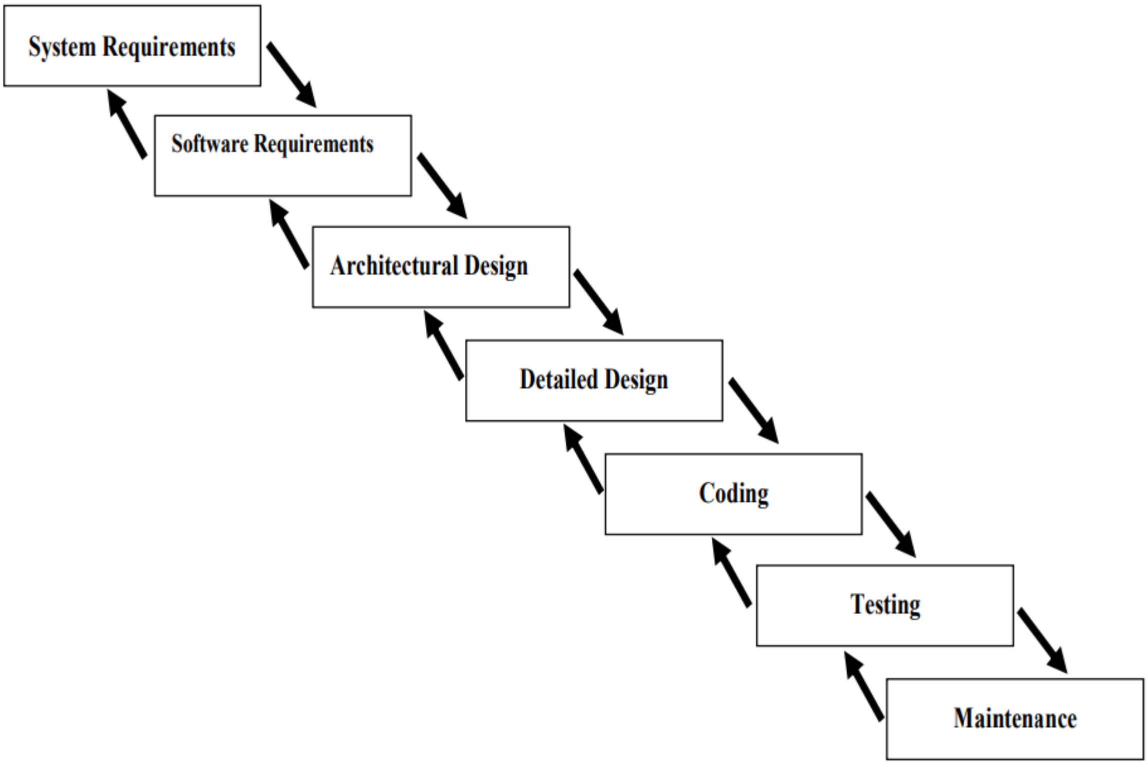
4.1 Selecting a methodology

When choosing a suitable methodology, it is necessary to ascertain the complexity of the solution, how many people will participate in the project and its nature. From the previous chapters is derived that the chatbot design focuses on meeting the defined requirements. However, further possible development of the chatbot must continue iteratively according to the analysis of conversations and user feedback. Future requirements may change from the client during development. These can be changes to the implementation of third-party API systems or even new requirements for integration into a new system. The chatbot solution designed in this work was developed only by the author; although, in practice there will presumably be a small team involved in the whole design that such a project would be developed by a smaller team even if only because of the multidisciplinary solution. In addition, one must acknowledge the fact that this work ends with finalized design; thus, it is necessary that the methodology selected is accommodating to any possible person, be it the author of this paper or anyone else, that could choose the result of this work as a base design for developing a chatbot.

After conducting some research, the waterfall model methodology was deemed to be most fitting for this concrete problem. Waterfall model is a sequential development process, in which development is viewed as constantly flowing stream or a falling waterfall through the phases of requirements, architectural design, detailed design, implementation, testing and operation and maintenance. The core idea of the waterfall model is that each phase needs to be properly finished and validated before another one starts. The model prospers from well-thought out detailed design. All the steps done in each phase need to be well documented and described, which consequently leads to new team members being able to join the project easily and quickly; the same applies to new teams that are handed the project over.

The waterfall model is often preferred due to its simplistic direct approach that is not hard to understand. The goals of each phase are clear, and the projects being developed with this methodology are usually very well structured. In figure 7 can be seen the diagram demonstrating the consequent steps of the waterfall model.

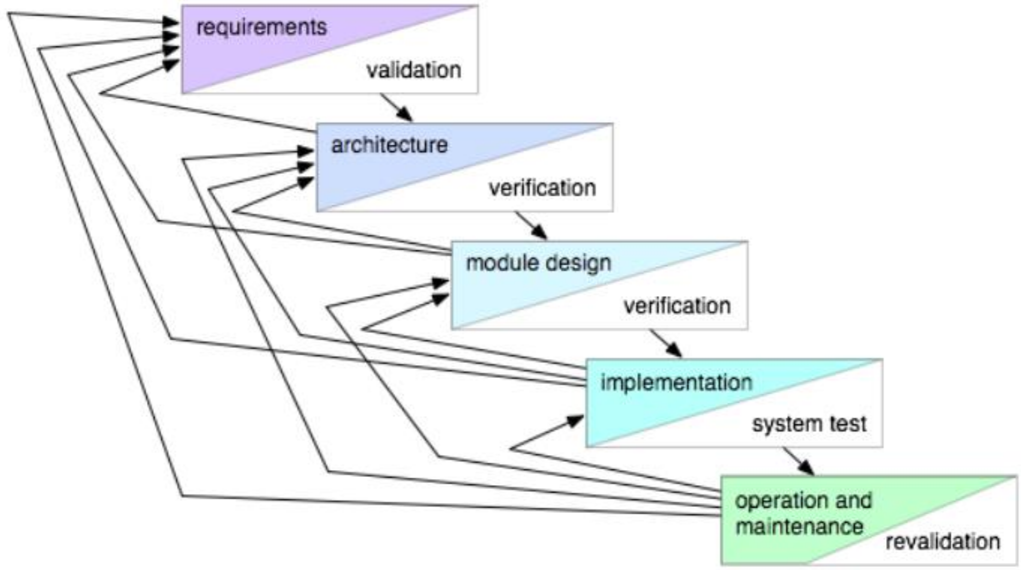
Figure 7: The waterfall model schema (Royce 1970)



Waterfall model is a classic in the area of software development and one of the oldest ones. Despite that, it is still commonly used in major companies or government projects (Munassar, Govardhan 2010), however, often not in its basic pure form since it hides considerable weaknesses such as high inflexibility of the process. The model also becomes excessively costly and extremely time-consuming in case of a significant mistake being discovered in the later stages of the project as to correctly follow the model one would have to repeat the concerning phase instead of just modifying the affected components. Moreover, the pure form of the model does not support implementing knowledge we might learn later in the process.

One of the modified versions of the waterfall model that diminishes the boldness of above-mentioned weaknesses is shown in figure 8. Each stage is open to modifications from latter stages if knowledge to prevent future bugs or to improve the functionality is discovered. Figure 8 demonstrates the feasible feedback paths that might occur in practice. This modified model is used as the methodology in consecutive portion of this thesis.

Figure 8: The waterfall model schema with feedback (Alspaugh 2019)



The design stage will be conducted based on the chatbot design process (Shevat 2017) of Amir Shevat. Mr Shevat is well experienced in product development and vastly familiar with chatbots as is the co-lead of community where are shared the best bot developing practices are shared. He designed many chatbots. He worked with google, Microsoft and currently is head of developer relations in Slack. (Shevat 2017) For these reasons is his method considered to be reliable and is therefore used as a reference for this work.

Originally is the process constructed of five steps; however, some of the steps assume that we are designing a chatbot for a common channel like Facebook Messenger. Since we are designing it to be run on a custom web widget the stages that are relevant for RecruBot are:

- Use case definition and exploration
- Conversation Scripting
- Bot building overview

All of these stages are to be inserted into detailed design phase.

4.2 Application of the waterfall model with feedback

Each one of the stages represents a crucial step in finalizing a software and each has clearly determined goals that need to be reached in order to successfully finish the stage. After the goals are reached, they need to be validated in order to move into the next phase. In the case that the goals are not found to be fulfilled, the phase needs to be repeated, the project cancelled, or the work of the previous stages corrected. In companies, validation is usually done either by a control committee, the customer or team leader.

4.2.1 Requirements analysis phase

To reach the goals in initial phase it is necessary to establish system requirements and aimed software requirements. All the needed components for our software to function should be listed including hardware requirement, all supporting software, the functions finalized project should have, their specifications and effects. Part of this phase is also determining any deadline the project would have. The result is typically requirement document but can replaced by any other proper documentation as long as it is clearly communicated.

For RecruBot, most of the requirements are already stated; thus, this phase properly formalizes them, reviews them and foreshadows a method or solution that can be utilized in order of meeting the requirements that is based on the section 3 of this thesis.

4.2.2 Architectural design phase

The objective of this phase is to determine how the architecture of the software will be organized. During this phase the crucial components must be identified and discovered how they need to interact; however, it does not focus on how each of the component is built individually. The output of this phase should be a documented architecture diagram or similar document as long as it communicates its purpose effectively.

For RecruBot's design the output of this phase will be a diagram where is demonstrated how all the components are communicating.

4.2.3 Detailed design phase

In this stage each of the component's functionalities are specified and a description specifying how each of them should be implemented and customized. In addition, what tools should be used to achieve the best finishing result.

For RecruBot there will be implemented the steps from the Amir Shevat's chatbot design process.

Use case definition and exploration

The first step is to describe the purpose and main functionalities that the chatbot we are designing need to have. The personality of the chatbot is also decided and described. This stage heavily impacts the image and branding of the chatbot.

Conversation Scripting

The output of this step is a conversation logic flowchart that includes all the expected conversation flows. Here is included entity and intent mapping together with the error handling. After finishing this stage, RecruBot is supposed to have all the dialog written down.

This step is considered to be primary for chatbots and is essential in setting RecruBot's tone and available services.

Bot building overview

In this stage is review what all other functionalities and components need to be incorporated for the chatbot to work. As a part of this stage is typically the evaluation and selection of available platform and channels to use the chatbot on; however, for RecruBot this will already be done as it is unnecessary to know for the structural design.

Next must be designed the supporting components that are crucial to enable the user to use the chatbot. This means that all the other components that ensure the connection to the presentation layer or the loading of vital data need to be designed here. The output of this stage varies depending on how the chatbot is devised. In case of RecruBot the output will be the design of how the database is communication with Watson Assistant, the communication of Watson Assistant and presentation layer and the structure of the database.

4.2.4 Implementation phase

The objective in this phase is to bring the design to the reality. All the needed components must be coded, customized and connected. Advisable practice is to implement the design component by component, although that is not always entirely possible because some components might dependable on other resulting in parallel implementation.

4.2.5 Testing phase

In this phase the implemented components that construct the software are systematically tested resulting into a report of issues that might need to be corrected or resolved. Besides searching for issues, in this stage is also tested if the software fulfils all the requirements. The aim of this stage is to successfully test all requirements with finding no problems, afterwards the waterfall can continue.

4.2.6 Operation and maintenance phase

In the final stage all the found issues must be closed and requirements met resulting in the software being ready for deployment to production. Subsequently the software must be provided with needed support and maintenance to ensure the continuous functionality of the deployed software.

5 Design of RecruBot

In the following chapter is documented the design process of RecruBot. The process is following the modified waterfall model that is described in section 4; respectively, the steps of the design correspond to the model's phases.

The design's approach is first to define the most general aspects of the chatbot and gradually delve into the more complicated ones. In other words, the approach is to start at the structure and work to the more specific functionality of chatbot dialog; then to design the structure of the database. At the end, the description in what way are all the components going to be communicating.

At the end of this phase, the whole RecruBot will be designed, and the development phases would be ready to start.

5.1 Requirements analysis

The goal of this phase is to summarize all the requirements that the chatbot must meet at the end of the fifth phase. In this work, the requirements will be evaluated based on how they are implemented into the design. Since the methodology is waterfall model with feedback, it is possible to revise the requirements in latter phases.

The requirements were already stated in chapter 2. In order to organize them, they are divided into two groups, functional requirements and non-functional requirements. This will help determine on which requirement to focus during the architecture phase and detailed design phase. The result of the division is displayed in table 2.

Table 2: List of requirements divided into functional and non-functional

id	Requirement name
	Functional
F1	Responding in natural language
F2	Understanding natural language
F3	Recognition of the intent
F4	Ability to extract entities
F5	Ability to inform about the open positions, the company and itself
F6	Pleasant corporate image
F7	Interesting personality
	Non-functional
N1	Database connection
N2	Accessible logs
N3	Simple portability
N4	Web user interface

All the knowledge needed to find ways of fulfilling the requirements are gathered in chapter 3 and will be utilized to analyze the requirements.

F1 requirement is depended on how the chatbot responses are written. Of course, this criterium is subjective as what sounds natural to one might not sound natural to the next. However, after reading the responses, any healthy human should be capable of assessing if

RecruBot writes up to some degree naturally or if its responses are awkwardly robotic with no feeling.

In order to make the RecruBot understand natural language, it needs to process it. The best solution would be to utilize one of the SaaS solutions from chapter 3.1.3. that already have an NLP component.

The first service that we can remove from consideration is Microsoft Luis since it does not offer all the services that are needed to construct a chatbot; it must be integrated into another platform, which increases the difficulty. Moreover, it is not as user friendly as other platforms as the intents and entities must be coded by hand.

The remaining services can be further filtered using the requirement N3, simple portability. What remains is IBM Watson Assistant and Google Dialogflow, as both offer a higher number of assisted integrations to major platforms like Facebook Messenger or Slack. Both are also easily used as an API for integrating into custom websites.

Both IBM Watson Assistant and Google Dialogflow are widely used; however, IBM Watson Assitant is better suited for RecruBot as its services are more corporation aimed, which brings along better support of fulfilling various policies such as GDPR[1]. In addition, unlike Google Dialogflow, IBM Watson Assistant supports Machine Learning based entities recognition that further assists in the development of the chatbot. In conclusion, IBM Watson Assistant best suits the general purpose of RecruBot and simplifies development the most.

By using the IBM Watson Assistant, the requirements F2, F3, and F4 will be met as part of the platform is a Dialog Manager and NLP component. Furthermore, IBM Watson Assistant gives access to its log of conversation as well as the general statistics, which concerns N2.

F5, F6, and F7 is also affected by the dialog and response design and is once again mostly subjective. Pleasant corporate image accompanies politeness and willingness to help. Any personality that is not painfully robotic and dull could be considered interesting. In order to increase the evaluation validity of these demands, a user testing method by Amir Shevat will be conducted to assess whether the conditions were met or not.

As already hinted while deciding on which cloud service to use, IBM Watson Assistant fulfills the N3 requirement by supporting integration to many other channels and by offering its own REST API to make putting one's chatbot on a custom page possible. Software development kits for NodeJS, swift, python, java, .NET, and Unity are already prepared on the platform. Furthermore, an integration of Botkit, an open-sourced framework for NodeJS, is also included. Botkit is released under the MIT Open Source license; therefore, is free to use even commercially. Thus, in order to create a web interface

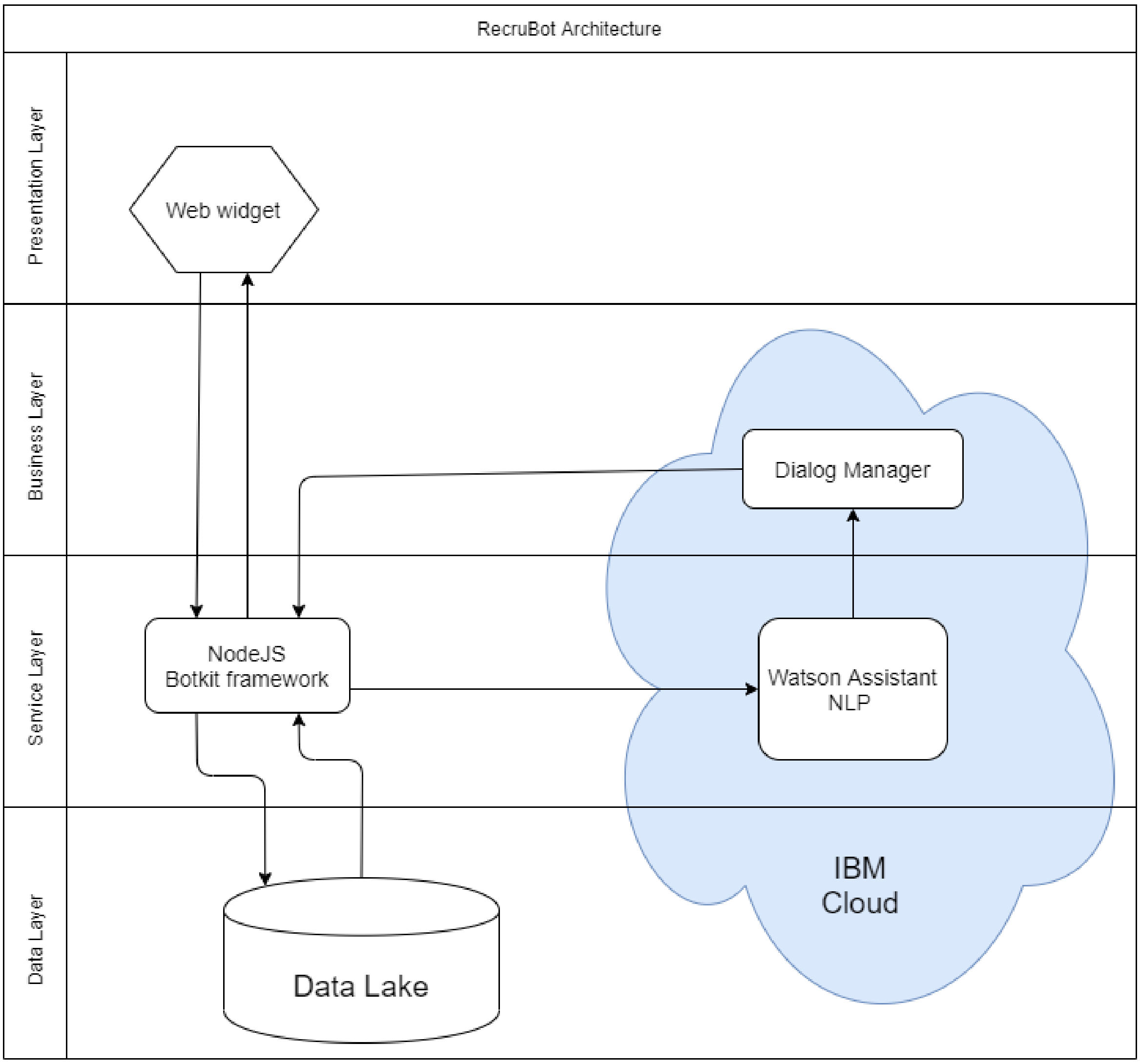
[1] General Data Protection Regulation

(N4), nodeJS with Botkit seems to be the ideal choice. Also, since Botkit supports the connection to most of the commonly used databases, it fulfills the N1 requirement as well.

5.2 Architectural design phase

In the previous chapter, it has been stated that the IBM Watson Assistant includes not only the NLP component but also the dialog manager as well as the logging functionality. Moreover, it supports the Botkit framework that decreases the complexity of creating a custom web widget that could be inserted into a webpage. In case further data for the functionality of RecruBot would be needed or specialized logging functions would have to be implemented, the Botkit also supports the communication with data stores.

Figure 9: Architecture diagram of RecruBot



The whole communication is demonstrated in figure 9. Users utilize the web widget to communicate with the chatbot that is possible thanks to the Botkit framework that sends their messages to the NLP processing. The NLP component identifies the intent and extracts the entities and sends the result to the dialog manager that bases the selection of the response on it. Both the NLP and dialog manager are services of IBM and are thus located in the IBM cloud. The response is then sent by the Botkit back to the widget that displays the message to the user. Through the Botkit framework is also connected the database where the needed data will be stored and loaded by the chatbot in order to answer queries about them.

In the designed architecture are incorporated all the vital parts that are needed to construct RecruBot that would satisfy all the predetermined requirements; therefore, the third phase might commence.

5.3 Detailed design

In this phase are designed and documented the core functionalities of RecruBot. To specify the conversation logic, dialogs flows, intent mapping, entity mapping, and web user interface designs follow.

5.3.1 Use case definition and exploration

The process of creating chatbot conversation that is followed here is described in the chapter 3.3.2. The process is further upgraded by some of the advice and steps from Amir Shevat's design process.

Three requirements are directly influenced by conversation design, F3, F4, F5, F6, and F7; therefore, with these in mind, the conversations are designed.

RecruBot's purpose

RecruBot's purpose is already defined by the requirements we have. What remains is to pick the correct ones and organize them. The resulting purpose is:

- Introducing the represented company
- Introducing itself
- Informing about the open positions
- Answering questions about the open positions

RecruBot's personality

From the requirements analysis, it is already stated that RecruBot must help create a pleasant corporate image, and its personality must not be considered boring. From these two requirements, some characteristics can be immediately dismissed. Such characteristics include but are not limited to rude, irresponsible, mendacious, or arrogant. Since it is a corporate chatbot, its personality traits should be closer to the business expectations of behavior; however, that alone would not fulfill the F7 requirement of an interesting persona.

Therefore, the corporate behavior must be combined with corporate acceptable behavior; for example, a funny businessman is not generally considered inappropriate.

The users that encounter RecruBot can be anyone. However, the ones' attention RecruBot wants to hold are primarily people that are presently unemployed, soon to be graduates, fresh graduates, or people looking for a change of employment. People that belong to this group are the RecruBot's targeted audience.

To create a RecruBot's persona with a targeted audience in mind that would be interesting while appropriate, it has been defined as follows:

An intelligent middle-aged man named Reggie that thinks positively and tries to motivate his surroundings to do better. He is very proud of the achievements of the company he is working for, and his greatest wish is for everyone to grow to their full potential. He loves animals, especially cats. Reggie is always in a good mood.

5.3.2 Conversation Scripting

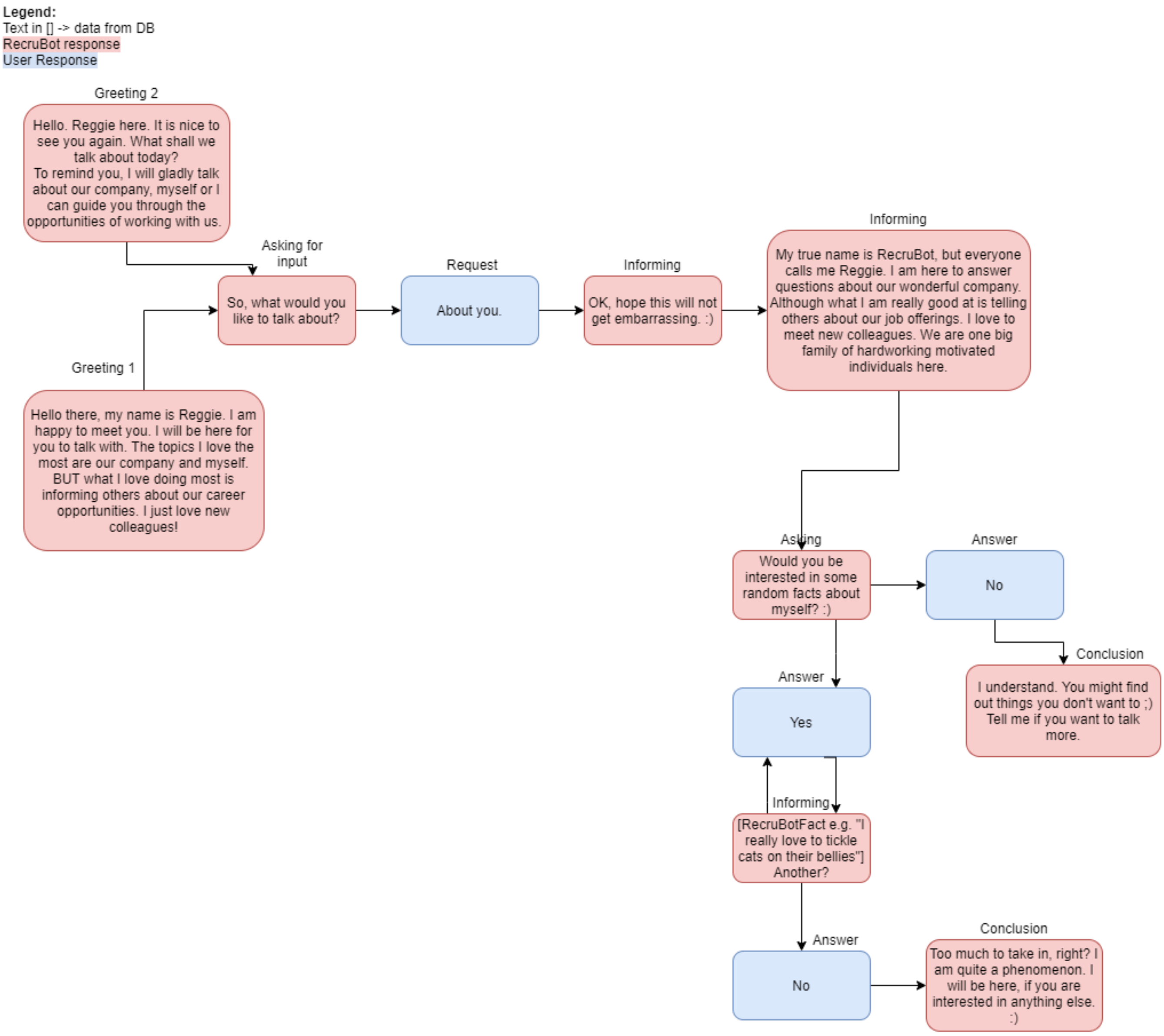
The three topics that RecruBot must cover in its dialog are:

- Self-introduction
- Company introduction
- Open positions information delivery

For each of these, there is a need to create a conversation diagram. Firstly, the general functional diagrams are made. Afterward, they are filled with the concrete responses of RecruBot. For each of the filled diagrams, we map out the intents and entities that are necessary to be taught to the RecruBot in development in order to make it recognize them.

The first diagram (figure 10) is the conversation flow that leads to chatbot introducing itself. It is very straightforward. After the user expresses that he would like to know more about RecruBot, it introduces itself in general; next gives the user the option to hear random facts about RecruBot. These facts purpose is to entertain by being truly random and possibly entertaining. An example of such a fact (that is also in the diagram) is that RecruBot likes to tickle cats' bellies. The facts are loaded from the database table recrubot_facts randomly.

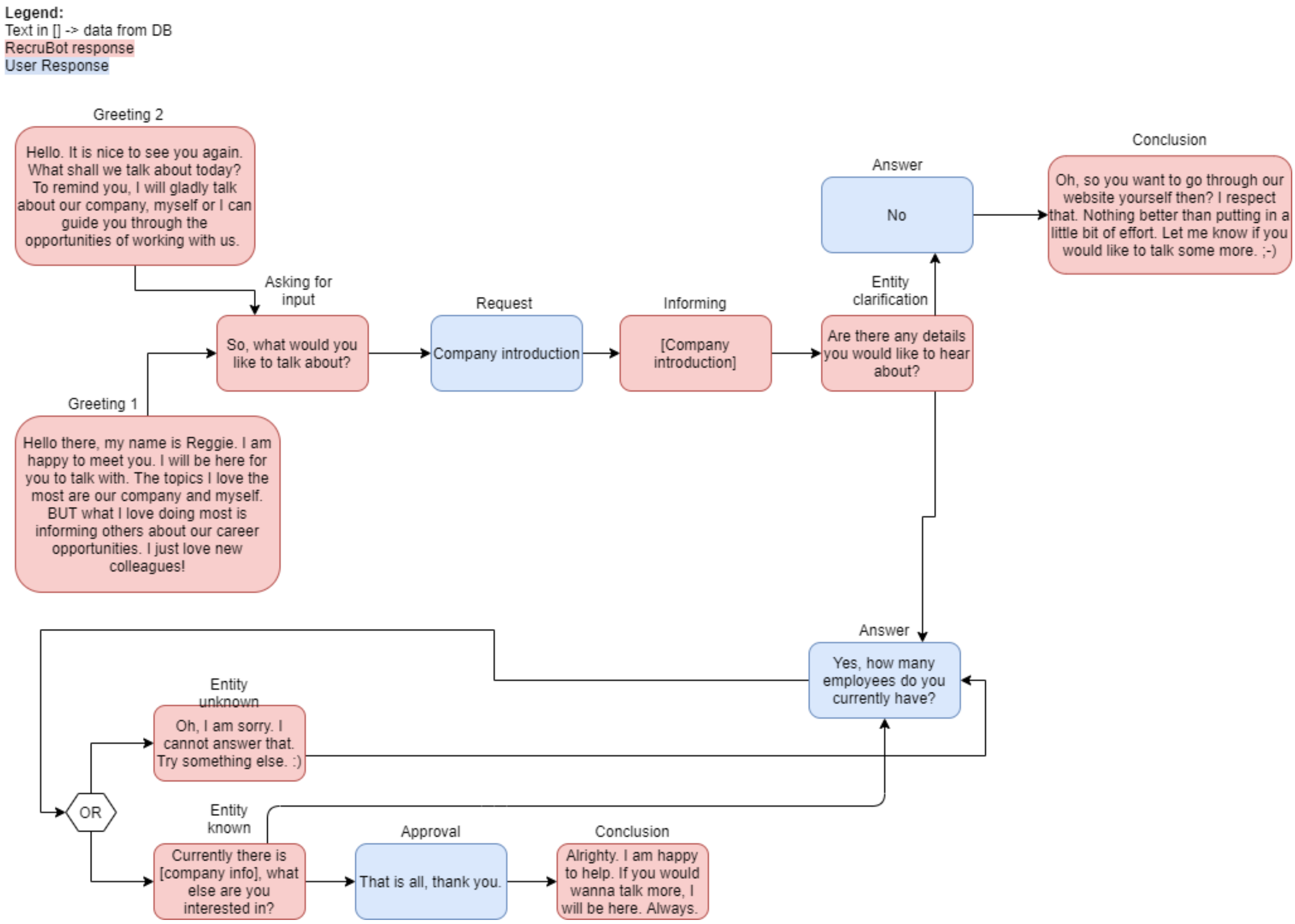
Figure 10: Conversation diagram - RecruBot, self-introduction



In figure 11 is the conversation diagram for the functionality of informing the user about the company chatbot represents. After recognizing the intent of requesting the company introduction, RecruBot responds with a general overview of the company. The overview is loaded from the database in order to make the update of it easier as the company's main achievements that it wants to broadcast the most might happen over time. Furthermore, since other data like the information about the open positions will also be updated and stored in the database, it would be impractical updating the company general introduction in another place.

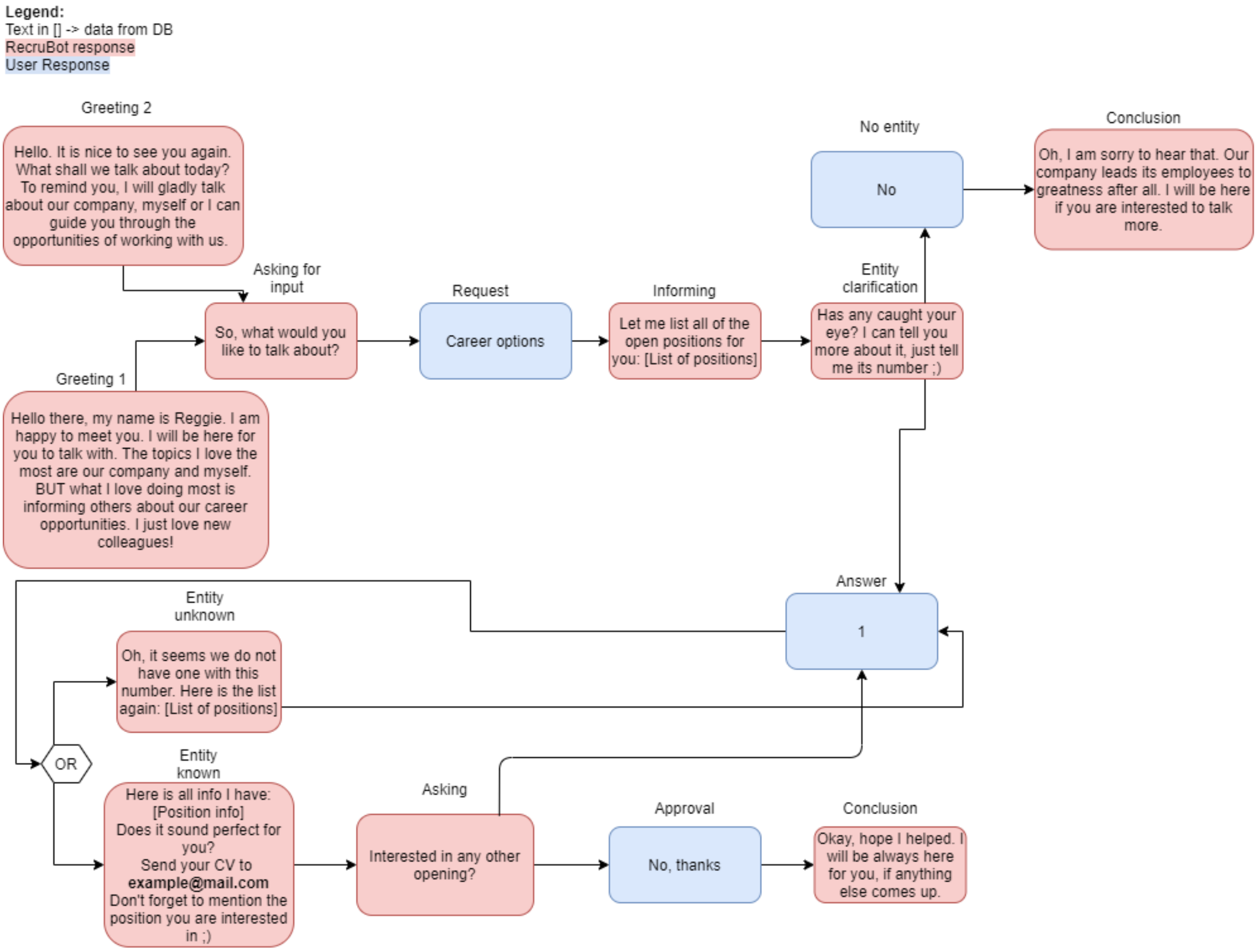
Followed by the introduction is the option for the user to ask questions about the company like "How many employees does your company have?", "Do you let cats to the offices?" "Who is the CEO?". The answers to these will be loaded from the database as well due to the its changeability over time, employees keep growing, policies change, and CEOs get replaced. Thus, a simpler way of updating the information is a great advantage to the sustainability of RecruBot.

Figure 11: Conversation diagram - RecruBot, company introduction



The third conversation diagram is shown in figure 12. It demonstrates how the logical flow of the main RecruBot's functionality is expected to work. It starts the same way as the other two. After the RecruBot receives the request of informing the user about the open positions, it responds by giving a numbered list of all the positions. The numbers correspond to the ID that the positions have allocated in the database table recrubot_openPositions. The user is then encouraged to give the number of the position he would like to hear the details of. If the given ID is allocated, he responds with the details of the position and how to apply for the position. The details given are dependent on what data is being kept in the connected database. The reason for keeping the data about the open positions in the database is quick and easy updatability as job openings are often changing or updating.

Figure 12: Conversation diagram - RecruBot, job openings information



All the conversations implement techniques that support the flow of the conversation; these include the reminders of what the chatbot can do and how to use it properly. An example of this can be seen in the RecruBot's response after the user chooses a number that is not in the list of open positions. RecruBot responses by listing the positions again, which hints to the use of what RecruBot needs in order to answer the user correctly. Another technique used is the open conclusion. In the last RecruBot's responses, it reminds the user that even though they ended their thread, RecruBot is still ready to listen to help again.

In addition, to make the dialog seem more natural, not all the RecruBot's responses are included in a single message but are sometimes divided into more. That way, the dialog seems more like a natural chat conversation. Also, the responses are written to not be bare but to contain filler words and discourse markers where it is appropriate, which causes RecruBot's language to be more humanlike. For instance, "So, what would you like to talk about?" the "so" is unnecessary, although without it the response would seem more robotic.

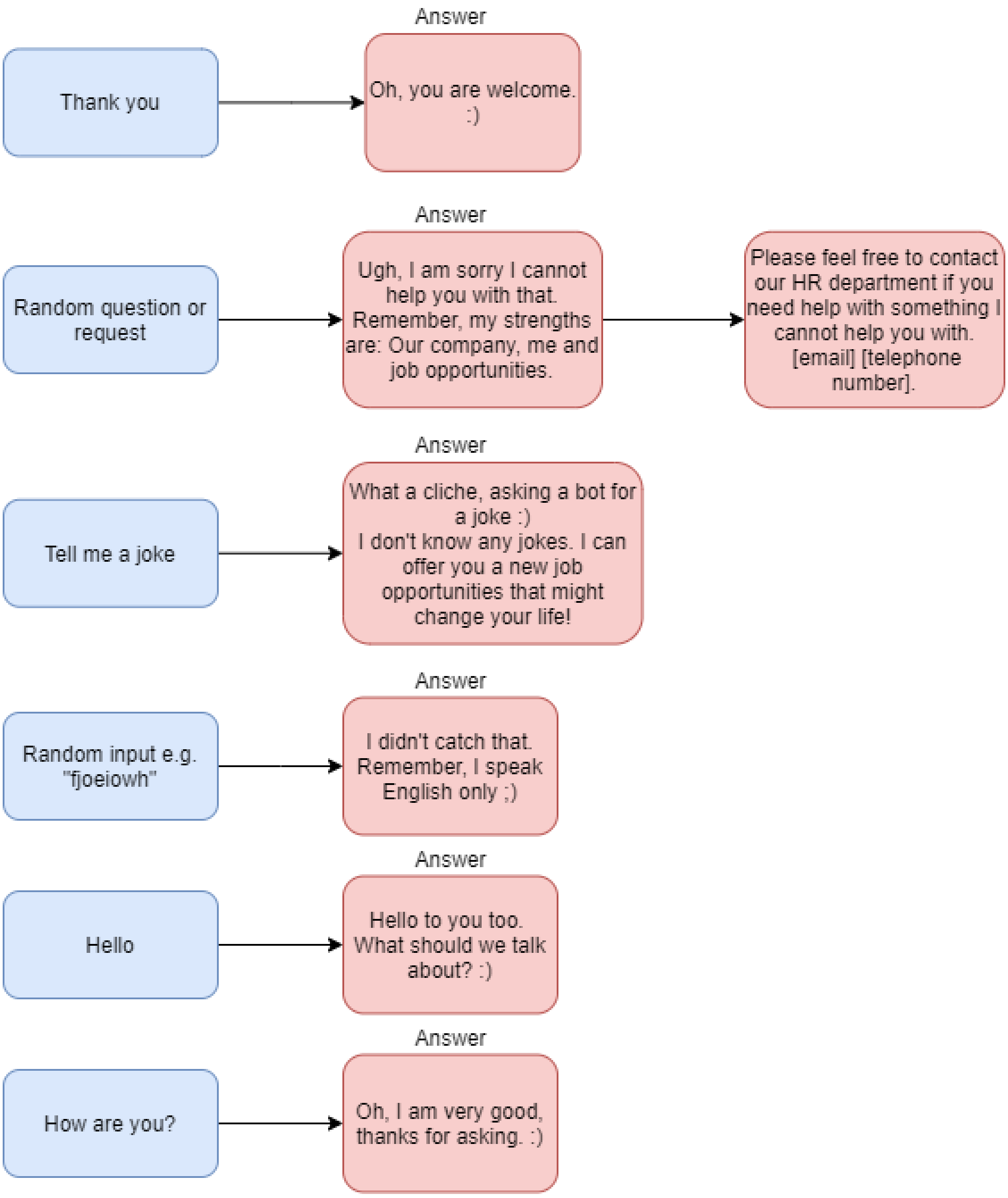
Error handling

Next, it is wise to prepare RecruBot for some general user messages that might come out of nowhere and are not part of the logical flow of the expected dialog that RecruBot is expecting. Teaching RecruBot to respond leads to the impression of a more advanced chatbot since it will prevent errors. Such user messages include:

1. Thanking the chatbot
2. Cursing the chatbot
3. Asking for a joke
4. Requesting or asking something that chatbot is not taught
5. Any other random input

In figure 13 are shown RecruBot's reactions to the general user messaging. One might notice that RecruBot is not to be taught how to respond to cursing; therefore, he will respond to it as if it was random input. The reason for not including the reaction to rude behavior is not to encourage such messages. As is believed, bullies are best repelled by ignorance.

Figure 13: RecruBot, conversation error handling



RecruBot's complete conversation logic flowchart is attached as annex I.

Intent mapping

Here the intents are be mapped out. In Watson Assistant, the intents are marked by the #. To create them, one needs to give Watson Assistant some example user inputs that will have the targeted intention since Watson Assistant uses machine learning for intent learning that is all that is needed to teach RecruBot the intent reliably.

Now follows a list of intents and their learning examples that are to be inserted into Watson Assistant.

#companyInfo
- → I wanna know more about the company.
- → Tell me more about the company.
- → Introduce me your company.
- → About your association…
- → What does your corporation do?
- → What business is this?
- → What company is this?
- → Let us talk about the company.

#employeesNumber
- → How many employees do you have?
- → How many is working for you?
- → How big is your company?
- → What is the size of your workforce?

#ceoName
- → Who is your CEO?
- → Chief executive officer?
- → Who is the big boss?
- → Boss of bosses?
- → Who is in charge?
- → Who is the most important officer?

#animalFriendlyOffice
- → Can I take a cat to the office?
- → Do you have animals in offices?
- → Are your offices animal friendly?
- → Are you animal friendly?
- → Do you let dogs in?

#howAreYou
- → How are you doing?
- → How are you?
- → What's up?
- → Watzaaaaap?

#hello
- → Hi
- → Hello
- → Ahoy
- → Hiya
- → Top of the mornin' to ya!
- → Howdy, partner!
- → What's kicking, little kitten?
- → Hello there
- → Yo!

#joke

→ Tell me a joke.
→ I want to hear a joke.
→ I want to laugh!

#thanks

→ Thank you.
→ Thanks.
→ Arigato.
→ Appreciate it.
→ Danke schön.

#openPositionList

→ Are you looking for new employees?
→ Can I work for your company?
→ I want to work with you.
→ You have any open positions?
→ Any job opportunities?
→ Let us talk about job opportunities.

#positionDetails

→ I want to hear more about position @positionNumber.
→ I am interested in one of the positions.
→ I need to learn more about a position.
→ @positionNumber.

#reggieInfo

→ Tell me about you?
→ You are interesting.
→ What are you?
→ Are you human?
→ I want to know more about you.
→ Let us talk about you.

#howToApply

→ How do I apply?
→ Who do I contact to apply?
→ Where do I send my CV?

#back

→ I don't want to talk anymore.
→ Have everything I need.
→ Let us end this.

Of course, the more examples the RecruBot will be given, the more knowledgeable he will get. Watson Assistant has the option of trying out the intents and entities in a test chat right in the dialog manager user face, where the messages that will not get properly identified can be marked as the proper intent or entities, making the chatbot even cleverer.

Entities mapping

The entities to be mapped out for the logical flow to work is only one, and that is the one to identify the position the user is interested in. In Watson Assistant, the entities are marked by @. They are defined by setting the values; to ensure proper entity identification, one may add synonyms to the values.

Watson Assistant already offers some general entities that are already mapped out, and to use them, they only need to be added. Their disadvantage, however, is that they cannot be edited, but one can explore how they are defined.

@positionNumber (numbers type, predefined in Watson Assistant)

→ No

Synonyms: not interested, nope, I don't want to.

The entities, as well as the intents, can be further edited while using the test chat in Watson Assistant.

5.3.3 Bot building overview

Since the conversations are designed what remains are the accompanying components that are necessary for RecruBot to be in working order. These components include database tables, user interface and the communication between the chatbot, data layer and presentation layer.

Database tables

From the designed conversations we see that the data that are to be loaded from the database is the information about open positions, company and random facts about RecruBot. Thus, the number of tables that will be containing the data is three.

In figure 14 are displayed all the tables that RecruBot will utilize. There are no relations between them; they are all alone standing. The design is purposely quite general; the reason for this is the fact that all the institutions that could be incorporating a new chatbot would already be using database. Therefore, it is very feasible that they would be unwilling to add another kind of database than they are already accustomed to as that would be highly redundant.

Figure 14: Database tables needed by RecruBot

recrubot_companyInfo	
Field Name	Data Type
comp_id	string
comp_info	string

recrubot_openPositions	
Field Name	Data Type
pos_id	int
pos_title	string
pos_descreption	string
pos_experience	string
pos_workHours	string
pos_details	string
pos_startDate	date

recrubot_facts	
Field Name	Data Type
fact_id	int
fact	string

The layout of each of the tables is quite simple. The first table, recrubot_companyInfo, is composed of only two fields, the comp_id and comp_info. The field comp_id corresponds to the name of the intent stored in RecruBot's dialog manager. The comp_info stores the answer, which will be returned. For instance, the user would ask, "Who is your CEO?", the intent here is #ceoName, thus from the database would be loaded the comp_info where comp_id = "ceoName" and sent to the user.

The table recrubot_openPositions contains pos_id that stands for position ID. The position ID is a number connected to each position. The rest of the fields contain information about the positions that will be employed to construct a reply in case a user would choose to hear more details about the position.

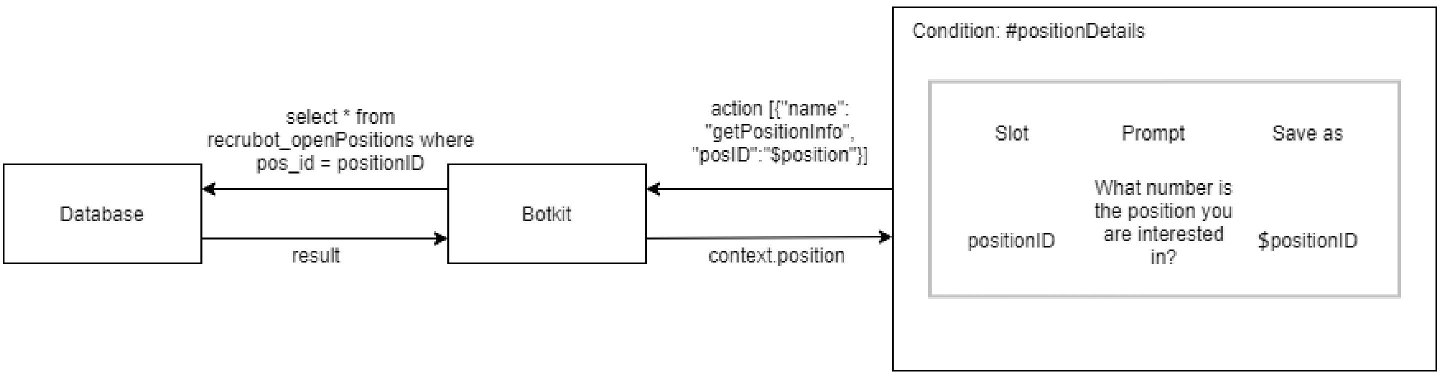
The last table is named recrubot_fact, where will be stored the responses to use when a user wants to hear a random fact bout RecruBot. The facts will be sent randomly, avoiding the previous one that was given. The field fact_id holds the number of the random fact while the field fact holds the actual random fact text.

All three tables' name starts with "recrubot" in order to easily identify their purpose of supporting the RecruBot.

Database communication

To obtain the needed data from the database, the so-called client action from dialog nodes will be utilized. A client action is a JSON message in a standardized format that can be sent out from Watson Assistant to Botkit. For the client actions to work, the backend must be taught to recognize the JSON format and, based on the action name, run the corresponding program. In the case of RecruBot, those programs will select needed data from the database and send it back to Watson Assistant enclosed in JSON message in the proper format. The process is shown in the diagram in figure 15; it is demonstrated on an example of loading the detail of a certain open position.

Figure 15: Client action diagram



The format of the client action JSON message can be seen in code 1. Array action defines what programs are to be called with what parameters. <actionName> corresponds to the name of the function, <parameter_value> is the value of parameter, there may be a bigger number of parameters or none; it depends on the function.

The <result_variable_name> references the JSON object that is to be returned by Botkit. The result is to be kept as a context variable in order to be displayed in Watson Assistant response or is available to be used by other response nodes.

Code 1: Client action JSON format (*Calling a client application* [no date])

```
{
  "context": {
    "variable_name" : "variable_value"
  },
  "actions": [
    {
      "name":"<actionName>",
      "parameters": {
        "<parameter_name>":"<parameter_value>",
        "<parameter_name>":"<parameter_value>"
      },
      "result_variable": "<result_variable_name>"
    }
  ],
}
```

Web widget communication

To make it possible for the users to talk to RecruBot, the chat messages need to be received and sent by the Watson Assistant. To achieve it, the Botkit includes a built-in chat server (benbrown 2018); by using it, the messages are directed to and from the Watson Assistant without the employment of any other third-party component.

In order to establish the connection, WebSockets will be used. Therefore, the possibility of two-way real-time communication becomes available. Connection using WebSockets stay open the whole time, which makes sending more than one message consecutively possible. The messages sent and delivered are in a JSON format where not only the content of the message is, but also metadata about the user and the connection. In addition, the processing of the request may take more time without the problem of encountering a timeout error.

The Botkit toolkit also offers a built-in webchat client that is already by default configured to support and manage the connection.

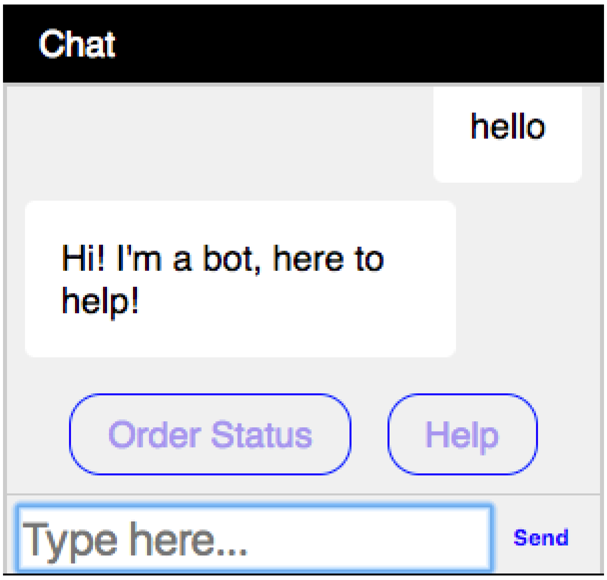
After opening the chat session, the chat client sends a hello or welcome_back client depending on if the user is connected for the first time or it is his repeated session. By this, RecruBot identifies if it is to use its Greeting 1 or Greeting 2. Furthermore, in the event or the session being disrupted, RecruBot can react by reconnect event or resending the messages that were not delivered.

Web widget user interface

As already mentioned, Botkit offers already configured web-based chat client that can be used freely. It is built with the help of HTML, CSS, and Javascript.

In figure 16 is seen the default design of the user interface; it is very simple and not very pretty. However, since the RecruBot is not designed with a specific company in mind, it would be redundant to spend an increased amount of time on the appearance of the front-end. If the design would be taken over and RecruBot developed, the design of the user interface would change depending on the institution.

Figure 16: User interface default design (benbrown 2018)



The chat window’s markup must be embedded into the corporation website in order to display it and use it.

6 Evaluation

In this chapter, the finalized design will be evaluated based on the requirements that were defined. Requirements numbered F1, F2, F5, F6, and F7 are to be evaluated based on the feedback from respondents that participated in interviews about these topics. This approach was chosen due to the subjective nature of these requirements. Thus, by evaluating them by more people makes it more valid. The rest of the requirements' fulfillment is judged based on the components' capabilities that are incorporated into the design.

Thereafter the design's sustainability, usability, and upgradability are discussed. Lastly, it is evaluated if the aim of this thesis was achieved.

6.1 Requirements evaluation

6.1.1 Interview study results; functional requirements

Each of the requirements will be discussed and afterward receive points based on the extent of fulfillment. The requirement can be considered met if the mark is at least 3/5.

The requirements F1, F2, F5, F6, and F7 are all quite subjective; therefore, a semi-structured interview was conducted in order to determine their fulfillment.

There were seven questions prepared in advance in order to make sure the important topics would be talked about. Additional questions were asked depending on where the conversation led. All interviewees were let to openly talk as long as they were expressing themselves to anything that is related to RecruBot.

The prepared questions were:

1. Does the chatbot respond in full sentences as a human operator would?
2. Is the chatbot able to inform about the company it represents?
3. Is the chatbot able to introduce itself?
4. Is the chatbot able to inform about the job opportunities in the company?
5. How do you find the chatbot's personality?
6. Is the chatbot appropriate to function in corporate environment?
7. Was the whole experience pleasant?

The interviews were conducted over MS Teams. Prior to the interview, they were left to play around with the testing chat in the Watson Assistant platform. The interviews were conducted in Czech; hence all the following citations from the responses are translated.

The respondents are all graduates or soon to be graduates of universities; the ages correspond to this fact being between 22 to 25. Their fields of study are varying. Two of the respondents' field is information technology; one studies biochemistry, one physics, one tourism, and the last one psychology. The level of their English was either B2, C1, or C2. There were seven respondents in total.

The responses to the first question, „Does the chatbot respond in full sentences as a human operator would?" were all positive. Two of the respondents noted that sometimes depending on what they write; the filler words are a little unsuitable; however, on the other hand, without them, the chatbot would sound too dull. This question mirrors the requirement F1 (responding in natural language). Since all the respondents' answer was positive only with occasional reservation, it receives a score of 4/5.

Then the next three questions were asked to find out if the RecruBot's is truly able to inform the users about itself, the company, and the open positions. All respondents had little or no trouble getting the chatbot to inform them about all three topics. Little trouble originated from different ways of asking the chatbot for the information. This can be remedied by giving the Watson Assistant more examples from which it is to recognize the correct intent since all respondents agreed that the chatbot is capable of giving the information the requirement F5 (ability to inform about the open positions, the company and itself receives 5/5.

The answers to the questions about the ReruBot's personality and appropriateness varied the most. Most of the respondents found chatbot's personality positive; their description included quotes such as „He is very pleasant to talk to, sometimes even funny…", „…it would be an interesting person, that kind of magical grandpa that everyone would like to visit…". Those who judged RecruBot's personality positively also agreed that it would be appropriate in the corporate environment. Two of the respondents' opinion was that it would not be appropriate in corporations as the RecruBot's responses were „…sometimes quite infantile…" or „…too familiar…". One complaint also included that the respondent felt as if the chatbot only cared about itself and its company since it was all it could talk about. This feeling was further supported when RecruBot was asked how he was but did not ask about the user's well-being back. One of the respondents also felt that in order to put the chatbot in the corporate environment, it would need to speak more formally; however, then admitted that it depended on what company would use it. All in all, the majority agreed that RecruBot was pleasant, quite humanlike, and appropriate for the corporate environment. Thus, the points for requirement F7 (interesting personality) is 3/5 and F6 (pleasant corporate image) 4/5.

The last question, "Was the whole experience pleasant?" was answered by all the respondents positively. None encountered bigger complications in the conversation and always were able to communicate their intention to the chatbot and understood the chatbot properly. After being asked if they had to "simple down" their messages to make the chatbot understand, all of the respondents answered with "no" or "very rarely. Therefore, the F2 (understanding natural language" gets 4.5/5

The respondents were able to communicate with the chatbot without any major difficulties, and Watson Assistant uses intent and entities recognition; from these two facts, it can be derived the RecruBot can effectively recognize the intent and identify the entities. Thus, both requirements F3 (recognition of the intents) and F4 (ability to extract entities) earn 5 points.

To summarize, all the respondents were happy or did not mind using the chatbot.

The limitation of the study is mainly the small sample that is caused by the interview being time demanding. The next limitation is that none of the respondents were native English speakers, which could skew their impressions of the chatbot's tone.

6.1.2 Non-functional requirements evaluation

What remains are the four non-functional requirements. These are N1 (database connection), N2 (Accessible logs), N3 (Simple portability) and N4 (web user interface).

The requirements N1, N3, and N4 are all achieved because of the utilization of Botkit, which not only supplies the web user interface, chat server but also supports connection to other messaging platforms using a simple API method. Furthermore, through it, is functional the loading of the data for RecruBot. However, since the design's functionality cannot be tested without its development, it is quite hard to rate it. On the other hand, the requirements are considered and based on the theoretical research should work. That is why the requirements N1, N3, and N4 receive 5/5.

The N2 requirement is part of IBM Watson Assistant and is widely used and functional, therefore it also receives 5/5.

6.2 General evaluation

Thereafter the design's sustainability, usability, and upgradability are discussed. Lastly it is evaluated if the aim of this thesis was achieved.

The sustainability of the design is superior since the finalized product that would be created based on this design would require little maintenance quite rarely. The chatbot can run non-stop without the need to maintain. The only need to assist it is when the data that it is using would change or an issue would develop. Thus, RecruBot would need to be updated. Also, all used components are supported and are still being upgraded.

The usability of the design is presently quite high as the design uses effective software and techniques. However, its usability will decrease with time as newer and more effective methods of achieving the same will be invented. Alternatively, the used components might get updated, and the information in this work might become outdated.

The process of upgrading RecruBot with new features and functionalities is not complicated as the Botkit framework supports various connection and the use of external services through the API methods.

Lastly, to determine if the aim of this work was achieved depends on fulfilling the requirements. In table 3 is the summary of the requirements and the final verdict of being met or not.

Table 3: Final evaluation of the requirements

id	Requirement name	Score	Fulfilled?
	Functional		
F1	Responding in natural language	4/5	Yes
F2	Understanding natural language	4.5/5	Yes
F3	Recognition of the intent	5/5	Yes
F4	Ability to extract entities	5/5	Yes
F5	Ability to inform about the open positions, the company and itself	5/5	Yes
F6	Pleasant corporate image	4/5	Yes
F7	Interesting personality	3/5	Yes
	Non-functional		
N1	Database connection	5/5	Yes
N2	Accessible logs	5/5	Yes
N3	Simple portability	5/5	Yes
N4	Web user interface	5/5	Yes

From the table is obvious that all the requirements were met, therefore the aim of this bachelor thesis is to be considered achieved.

Conclusion

The main topic of this thesis was the design of an HR chatbot, which was also its aim. At the beginning, the history of artificial intelligence and chatbots was introduced. Afterwards the requirements needed to be met in order to achieve this work's aim were defined in chapter 3, this served as guide to what knowledge needs to be gathered and explored; this is realized in the third chapter were all the necessary information, components and techniques for the final design were discussed.

In the following fourth chapter was explained what methods were used to design RecruBot together with why they were chosen. The whole aim of this thesis was achieved in the fifth chapter where the whole design process took place. At the end of the chapter, all the components were laid out.

The last chapter is the evaluation of the design. To help with the evaluation of the more subjective aspects, interviews were conducted to find out how RecruBot affects users. All the requirements were analyzed, and it was decided if they were met or not. Through this, it was verified if the aim of the thesis was truly achieved. Lastly, the sustainability, usability, and upgradability of the design was discussed.

List of references

A Breakdown of Chatbot Architecture And How It Works, 2019. *Enterprise Bot Manager* [online]. [Accessed 13 April 2020]. Available from: https://www.enterprisebotmanager.com/chatbot-architecture-and-how-they-work/

ALSPAUGH, Yhomas A., 2019. Software Process Models. *Thomas A. Alspaugh* [online]. 11 May 2019. [Accessed 24 April 2020]. Available from: https://www.thomasalspaugh.org/pub/fnd/softwareProcess.html#Royce1970-mdls

APPLIEDAI, 2018. Natural Language Platforms: In-Depth Guide [2020 update]. *appliedAI* [online]. 4 April 2018. [Accessed 7 April 2020]. Available from: https://blog.aimultiple.com/natural-language-platforms/

ARTHUR, Diane, 2006. *Recruiting, Interviewing, Selecting & Orienting New Employees*. 4th ed. New York: AMACOM. ISBN 978-0-8144-0861-2. HF5549.5.R44 A75 2006

ASHER, Natali, 2017. *A Warmer Welcome – Application of a Chatbot as a Facilitator for New Hires Onboarding*. 2017. Linnaeus University. Faculty of Technology. Department of Media Technology.

BENBROWN, 2018. howdyai/botkit-starter-web. *GitHub* [online]. 2018. [Accessed 9 May 2020]. Available from: https://github.com/howdyai/botkit-starter-web

BLACK, Hillary, 2020. What is Conversation Design, and How to Design Your Chatbot. *Medium* [online]. 15 April 2020. [Accessed 20 April 2020]. Available from: https://medium.com/swlh/what-is-conversation-design-and-how-to-design-your-chatbot-3754f04ab1e7

Calling a client application, [no date]. [online]. [Accessed 9 May 2020]. Available from: https://cloud.ibm.com/docs/assistant?topic=assistant-dialog-actions-client&fbclid=IwAR2cwjgxlmQcFV_zVN9c5A7XHo3qEslUNHqdNJigzovV8ymhDtyU0YScAAg

DASAGRANDHI, Charan Sai, [no date]. Chatbot Development: Designing Dialog Management. [online]. [Accessed 18 April 2020]. Available from: https://blog.vsoftconsulting.com/blog/chatbot-development-designing-dialog-management

FRANKENFIELD, Jake, 2019. Chatbot. *Investopedia* [online]. 2019. [Accessed 24 March 2020]. Available from: https://www.investopedia.com/terms/c/chatbot.asp

GARBADE, Michael J., 2018. A Simple Introduction to Natural Language Processing. [online]. 15 October 2018. [Accessed 30 March 2020]. Available from: https://becominghuman.ai/a-simple-introduction-to-natural-language-processing-ea66a1747b32

GOYAL, Shipra Goyal and RATTAN, Dhavleesh, 2017. Natural Language Processing Its Types. *International Journal of Advanced Research in Computer Science; Udaipur* [online]. September 2017. Vol. 8, no. 8. [Accessed 2 April 2020]. Available from: http://search.proquest.com/docview/1953779933/abstract/6C9732C288A44F0BPQ/1

GUENOLE, Nigel and FEINZIG, Sheri, 2018. The Business Case for AI in HR. . November 2018. P. 36.

GÜZELDERE, Güven and FRANCHI, Stefano, 1995. dialogues with colorful personalities of early ai. [online]. 24 July 1995. [Accessed 25 March 2020]. Available from: https://web.stanford.edu/group/SHR/4-2/text/dialogues.html

How Chatbots are transforming Wall Street and Main Street banks?, 2019. [online]. [Accessed 25 March 2020]. Available from: https://marutitech.com/chatbots-transforming-wall-street-main-street-banks/

JASSOVA, Barbora, 2020. Natural Language Processing Chatbot Explained. *Landbot.io* [online]. 2 January 2020. [Accessed 1 April 2020]. Available from: https://landbot.io/blog/natural-language-processing-chatbot/

JOSHI, Naveen, 2019. Recruitment Chatbots: Is The Hype Worth It? *Forbes* [online]. 9 February 2019. [Accessed 25 March 2020]. Available from: https://www.forbes.com/sites/cognitiveworld/2019/02/09/recruitment-chatbot-is-the-hype-worth-it/

JURAFSKY, Dan and MARTIN, James H., 2019. Speech and Language Processing. [online]. 2019. [Accessed 15 April 2020]. Available from: https://web.stanford.edu/~jurafsky/slp3/

KHAN, Roshan, 2017. Standardized Architecture for Conversational Agents a.k.a. ChatBots. *International Journal of Computer Trends and Technology*. 25 August 2017. Vol. 50, no. 2, p. 114–121. DOI 10.14445/22312803/IJCTT-V50P120.

LIEBL, Leah, 2019. 6 Steps for Creating a Smooth Chatbot Conversation Flow. *Userlike Live Chat* [online]. 24 September 2019. [Accessed 20 April 2020]. Available from: https://www.userlike.com/en/blog/chatbot-conversation-flow

MCCORDUCK, Pamela, 2004. *Machines who think: a personal inquiry into the history and prospects of artificial intelligence*. 25th anniversary update. Natick, Mass: A.K. Peters. ISBN 978-1-56881-205-2.
Q335 .M23 2004

MONSTERS, Data, 2019. 50 Chatbot Platforms: 2019 Edition. *Medium* [online]. 29 November 2019. [Accessed 3 April 2020]. Available from: https://medium.com/@datamonsters/50-chatbot-platforms-2019-edition-e6be022a7e0e

MUNASSAR, Nabil Mohammed Ali and GOVARDHAN, A, 2010. A Comparison Between Five Models Of Software Engineering. . 2010. Vol. 7, no. 5, p. 9.

NADEAU, David and SEKINE, Satoshi, 2006. A survey of named entity recognition and classification. . 2006.

NOVET, Jordan, 2017. Everyone keeps talking about A.I.—here's what it really is and why it's so hot now. *CNBC* [online]. 17 June 2017. [Accessed 24 March 2020]. Available from: https://www.cnbc.com/2017/06/17/what-is-artificial-intelligence.html

NOVÝ, Marek, 2019. CzechBot - chatbot pro obce. [online]. 2019. [Accessed 30 March 2020]. Available from: https://vskp.vse.cz/76606_czechbot___chatbot_pro_obce

PHILLIPS, Casey, 2018. The 3 Types of Chatbots & How to Determine the Right One for Your Needs. *Medium* [online]. 30 April 2018. [Accessed 31 March 2020]. Available from: https://chatbotsmagazine.com/the-3-types-of-chatbots-how-to-determine-the-right-one-for-your-needs-a4df8c69ec4c

ROYCE, Winston W., 1970. Managing the development of large software systems: concepts and techniques. In: *Proceedings of the 9th international conference on Software Engineering*. 1970. p. 328–338.

SHEVAT, Amir, 2017. *Designing Bots: Creating Conversational Experiences*. 1 edition. Beijing ; Boston: O'Reilly Media. ISBN 978-1-4919-7482-7.

SINGH, Abhishek, RAMASUBRAMANIAN, Karthik and SHIVAM, Shrey, 2019. *Building an Enterprise Chatbot: Work with Protected Enterprise Data Using Open Source Frameworks* [online]. Berkeley, CA: Apress. [Accessed 17 April 2020]. ISBN 978-1-4842-5033-4. Available from: http://link.springer.com/10.1007/978-1-4842-5034-1

STATISTA, 2020. Most popular messaging apps 2019. *Statista* [online]. 2020. [Accessed 19 April 2020]. Available from: https://www.statista.com/statistics/258749/most-popular-global-mobile-messenger-apps/

THOMSON, Clive, 2007. I Chat, Therefore I Am... *Discover Magazine* [online]. 3 May 2007. [Accessed 25 March 2020]. Available from: https://www.discovermagazine.com/technology/i-chat-therefore-i-am

Top 5 NLP Chatbot APIs to Make Your First Conversational Chatbot, 2018. *Data Science Learner* [online]. [Accessed 7 April 2020]. Available from: https://www.datasciencelearner.com/top-5-nlp-chatbot-apis-conversational/

What is a Chatbot? - Definition from Techopedia, [no date]. *Techopedia.com* [online]. [Accessed 24 March 2020]. Available from: https://www.techopedia.com/definition/16366/chatterbot

What is a Chatbot? Definition & Meaning | Botpress, [no date]. [online]. [Accessed 24 March 2020]. Available from: https://botpress.com/learn/what-and-why

YSE, Diego Lopez, 2019. Your Guide to Natural Language Processing (NLP) - Towards Data Science. [online]. 15 January 2019. [Accessed 1 April 2020]. Available from: https://towardsdatascience.com/your-guide-to-natural-language-processing-nlp-48ea2511f6e1

Annexes

Here is attached the whole RecruBot's conversation logic flowchart

Annex A: RecruBot dialog logic flowchart

Legend:
Text in [] -> data from DB
RecruBot response
User Response